

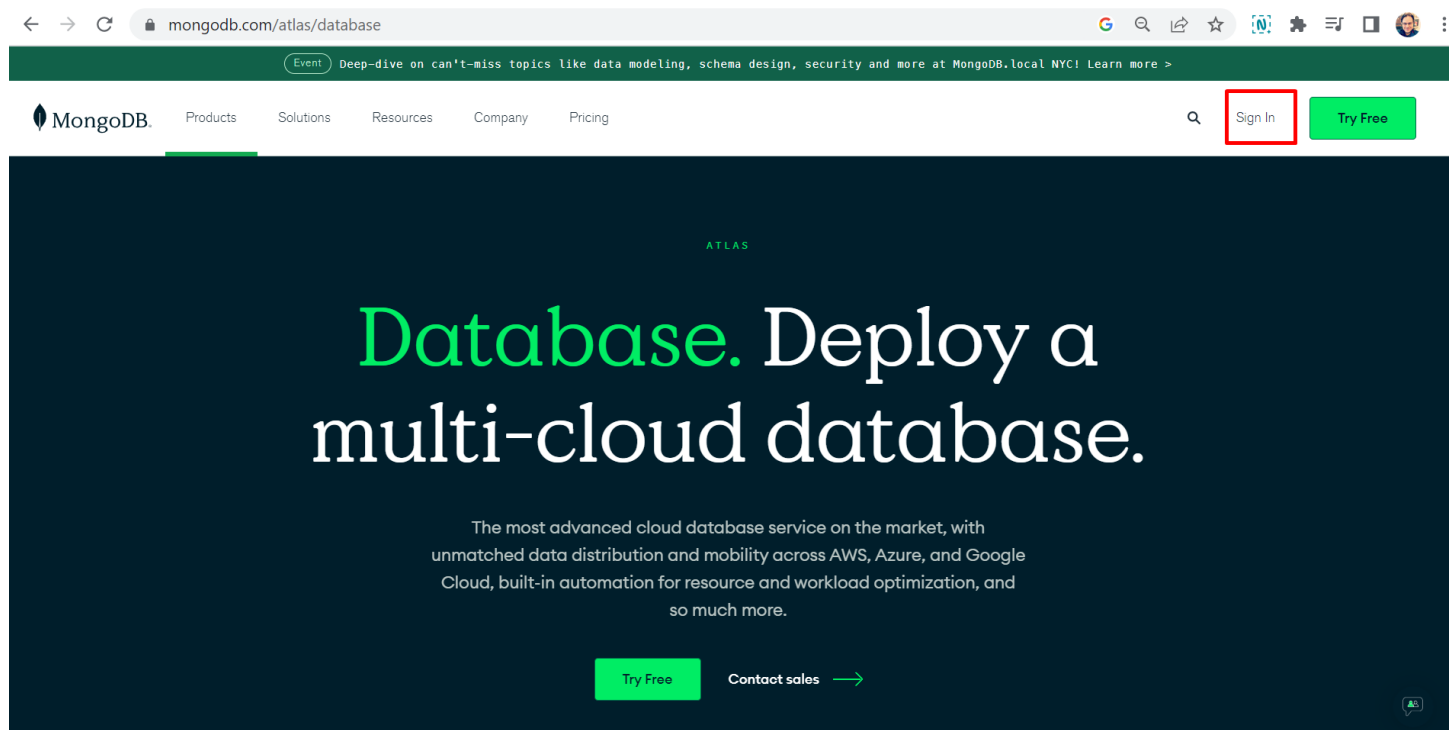
Aula 4 – API REST com Node JS e MongoDB

Objetivo da aula:

- Criando Banco de Dados no MongoDB Atlas.
- Instalando o Mongoose
- Fazendo conexão com MongoDB pelo Mongoose
- Organizando rotas
- Enviando dados para o MongoDB via API no Postman

Na última aula montamos criamos a estrutura base de um servidor Back-End para API, porém apenas testamos o CRUD com um JSON local, vamos agora utilizar o MongoDB e permitir o uso de CRUD por meio de API REST.

01 – Vamos criar uma conta no MongoDB Atlas, acesse: <https://www.mongodb.com/atlas/database> e clique em Sign In. O Atlas é uma cloud gratuita que o MongoDB fornece para testes de banco de dados não relacional.



02 – Após o login ou cadastro de conta, você deverá ver uma tela similar a essa. Vamos criar um New Projeto.

The screenshot shows the MongoDB Atlas interface. The browser address bar displays the URL: `cloud.mongodb.com/v2#/org/63580325dcb5a57507206aac/projects`. The top navigation bar includes the Atlas logo, a dropdown menu for 'Marcio's Org...', and links for 'Access Manager' and 'Billing'. On the right, there are links for 'All Clusters', 'Get Help', and a user profile dropdown for 'Marcio'.

The left sidebar is titled 'ORGANIZATION' and lists several options: 'Projects' (highlighted), 'Alerts', 'Activity Feed', 'Settings', 'Integrations', 'Access Manager', 'Billing', 'Support', and 'Live Migration'.

The main content area is titled 'Projects' and shows a table with the following data:

Project Name	Database Deployments	Users	Teams	Alerts	Actions
Project 0	0 Deployments	2 Users	0 Teams	0 Alerts	... Delete

Below the table, the system status is 'All Good' and the last login is '186.210.1.71'. The footer includes copyright information for MongoDB, Inc. and links to 'Status', 'Terms', 'Privacy', 'Atlas Blog', and 'Contact Sales'.

A red rectangular box highlights the 'New Project' button in the top right corner of the main content area.

03 – Coloque um nome para seu projeto.

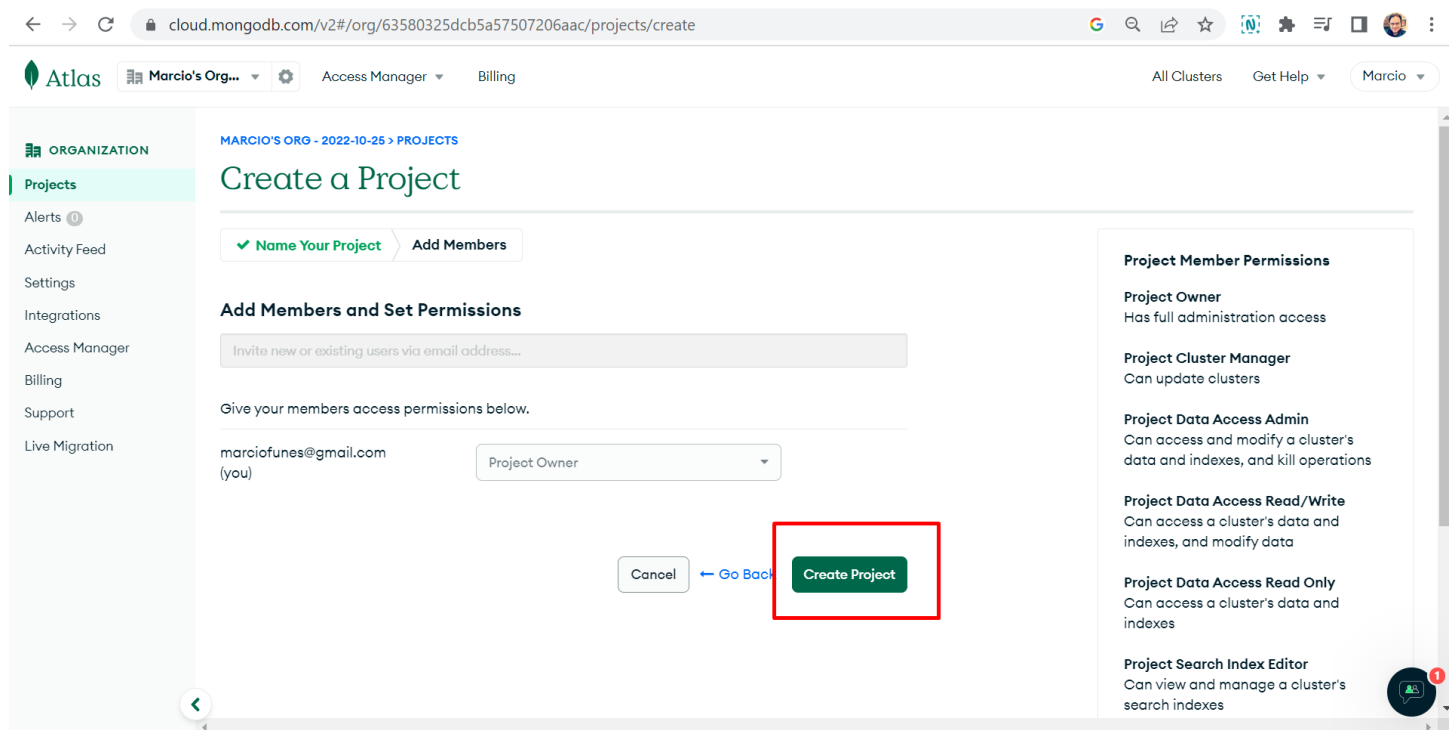
The screenshot displays the MongoDB Atlas web interface for creating a new project. The browser address bar shows the URL: `cloud.mongodb.com/v2#/org/63580325dcb5a57507206aac/projects/create`. The top navigation bar includes the Atlas logo, the organization name 'Marcio's Org...', and links for 'Access Manager' and 'Billing'. On the right, there are links for 'All Clusters', 'Get Help', and a user profile dropdown for 'Marcio'.

The left sidebar, under the 'ORGANIZATION' header, lists various navigation items: 'Projects' (highlighted), 'Alerts', 'Activity Feed', 'Settings', 'Integrations', 'Access Manager', 'Billing', 'Support', and 'Live Migration'.

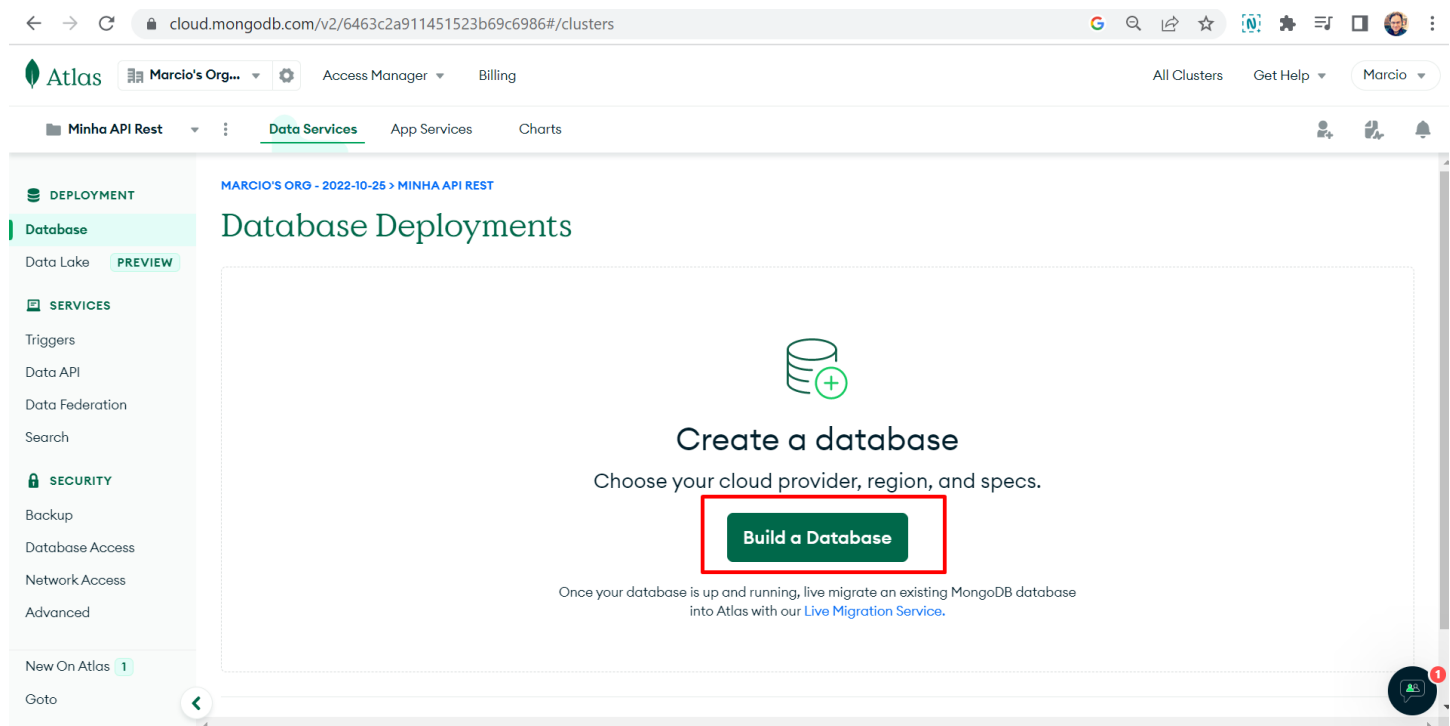
The main content area is titled 'Create a Project' and shows the progress of the setup. The current step is 'Name Your Project', indicated by a highlighted tab. Below this, a message states: 'Project names have to be unique within the organization (and other restrictions)'. A text input field contains the text 'Minha API Rest'. At the bottom right of the input area are two buttons: 'Cancel' and 'Next'.

At the bottom of the page, a system status bar indicates 'System Status: All Good' and 'Last Login: 186.210.1.71'. Below this is a copyright notice: '©2023 MongoDB, Inc.' followed by links for 'Status', 'Terms', 'Privacy', 'Atlas Blog', and 'Contact Sales'. A small circular notification icon with a red badge is visible in the bottom right corner.

04 – Não será necessário nenhuma configuração nessa tela, clique em Create Project.



05 – Projeto criado, vamos agora criar um Banco de Dados.



06 – Escolha a opção gratuita M0

cloud.mongodb.com/v2/6463c2a911451523b69c6986#/clusters/starterTemplates

 **MongoDB**

Deploy your database

Use a template below or set up [advanced configuration options](#). You can also edit these configuration options once the cluster is created.

M10 **\$0.08/hour**

For production applications with sophisticated workload requirements.

STORAGE	RAM	vCPU
10 GB	2 GB	2 vCPUs

SERVERLESS **\$0.10/1M reads**

For application development and testing, or workloads with variable traffic.

STORAGE	RAM	vCPU
Up to 1TB	Auto-scale	Auto-scale

M0 **FREE**

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
512 MB	Shared	Shared

FREE

[Access Advanced Configuration](#)

Create

Free forever! Your M0 cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

[I'll deploy my database later](#)



07 – Após alguns minutos o Cluster do servidor em nuvem foi criado, vá em Database para ver as informações do banco de dados.

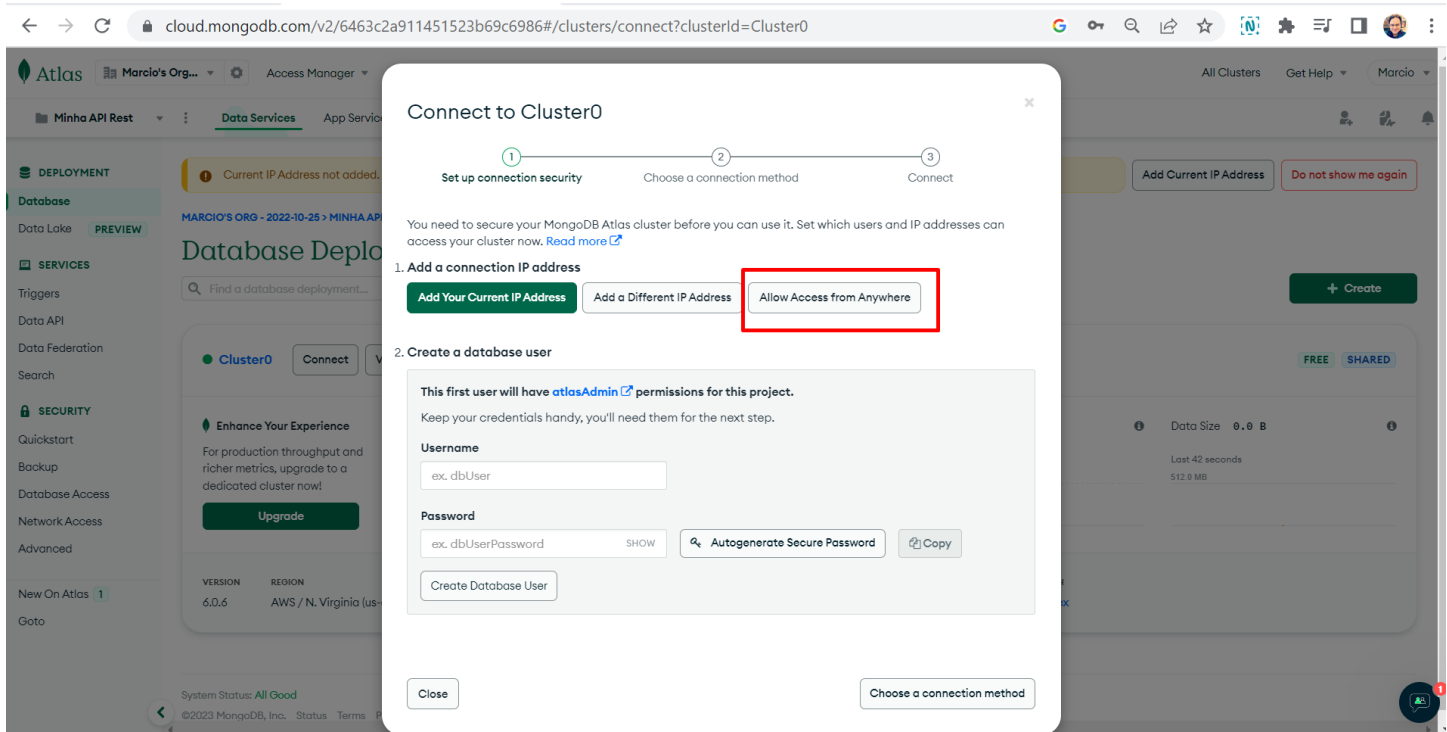
The screenshot shows the MongoDB Atlas interface. The left sidebar has 'Database' highlighted. The main content area is titled 'Database Deployments'. A red box highlights the 'Cluster0' deployment card. The card shows a 'Connect' button, 'View Monitoring', and 'Browse Collections' options. Below these are several performance metrics: 'Enhance Your Experience' with an 'Upgrade' button, 'Connections' (0), 'In/Out B/s' (0.0), and 'Data Size' (0.0 B). At the bottom of the card is a table with deployment details.

VERSION	REGION	CLUSTER TIER	TYPE	BACKUPS	LINKED APP SERVICES	ATLAS SQL	ATLAS SEARCH
6.0.6	AWS / N. Virginia (us-east-1)	M0 Sandbox (General)	Replica Set - 3 nodes	Inactive	None Linked	Connect	Create Index

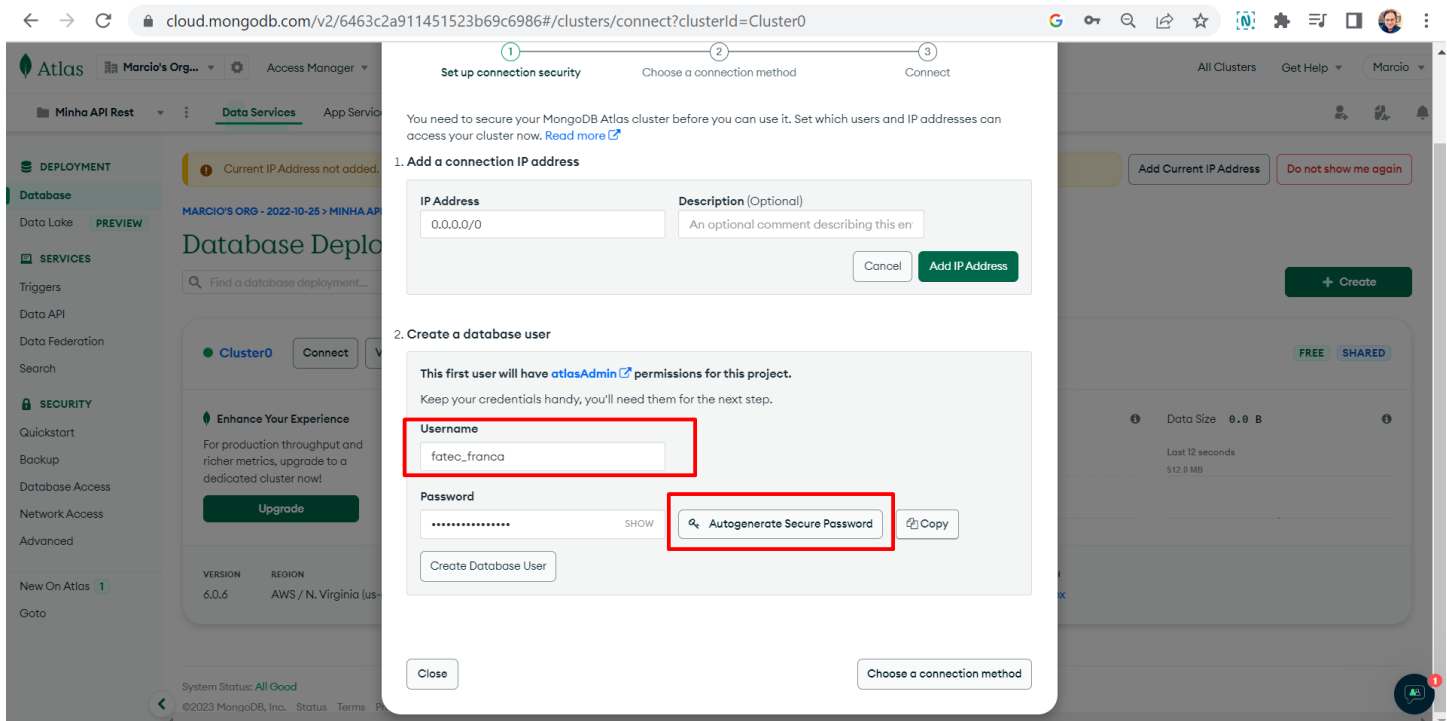
08 – Agora queremos que nosso back-end consiga se conectar a este Banco de Dados, clique em Connect.

This screenshot is identical to the previous one, but with a red box and an arrow pointing to the 'Connect' button on the 'Cluster0' deployment card, indicating the next step in the process.

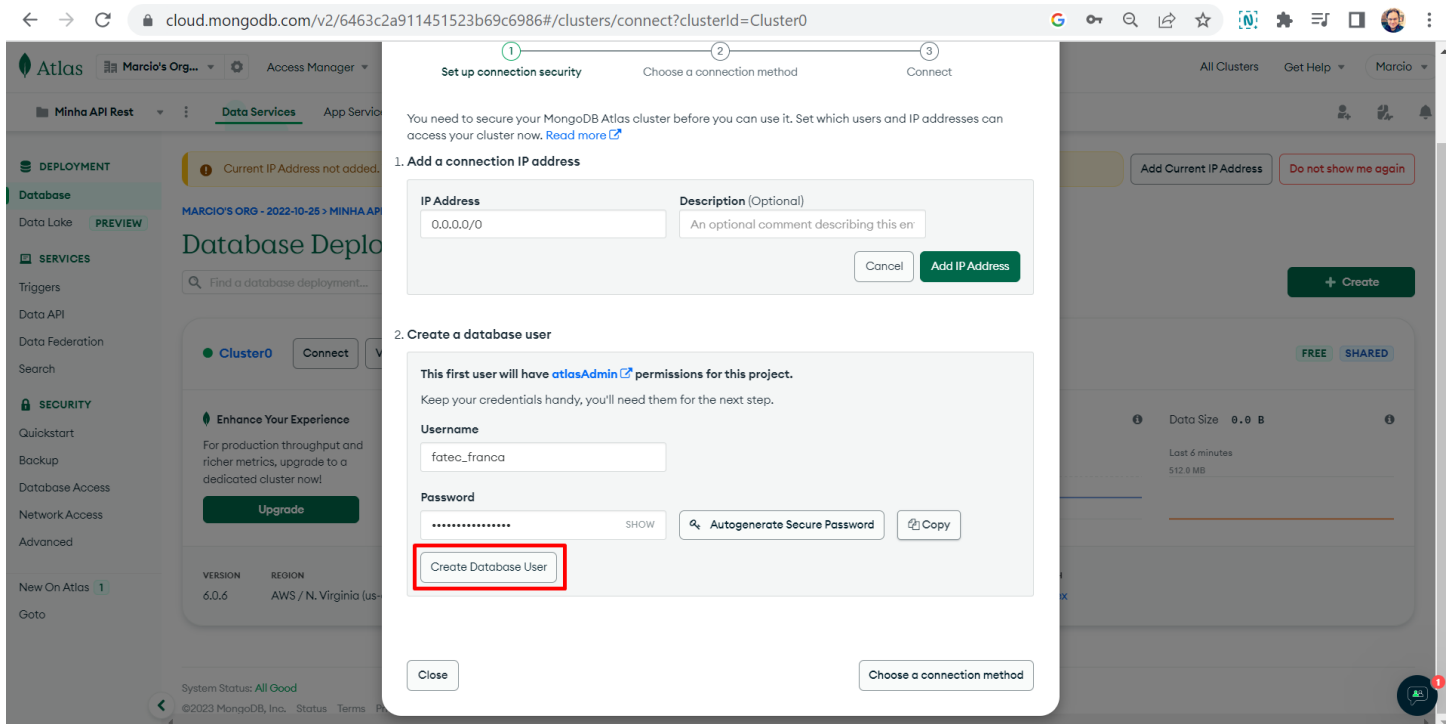
09 – Selecione “Allow Access from Anywhere”.



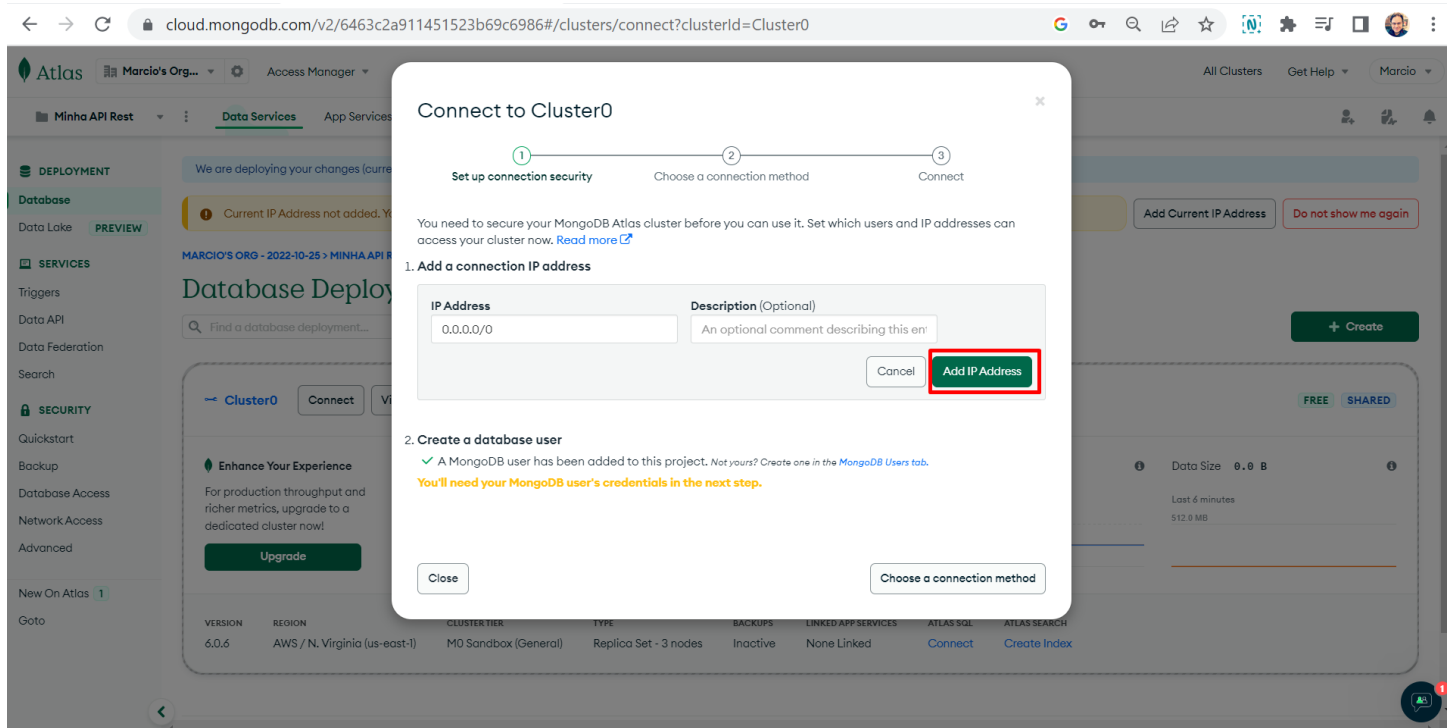
10 – Agora coloque um Username e uma senha, você pode solicitar que o Atlas gere uma senha. Anote esse usuário e senha em algum lugar pois logo usaremos em nossa aplicação para realizar a conexão.



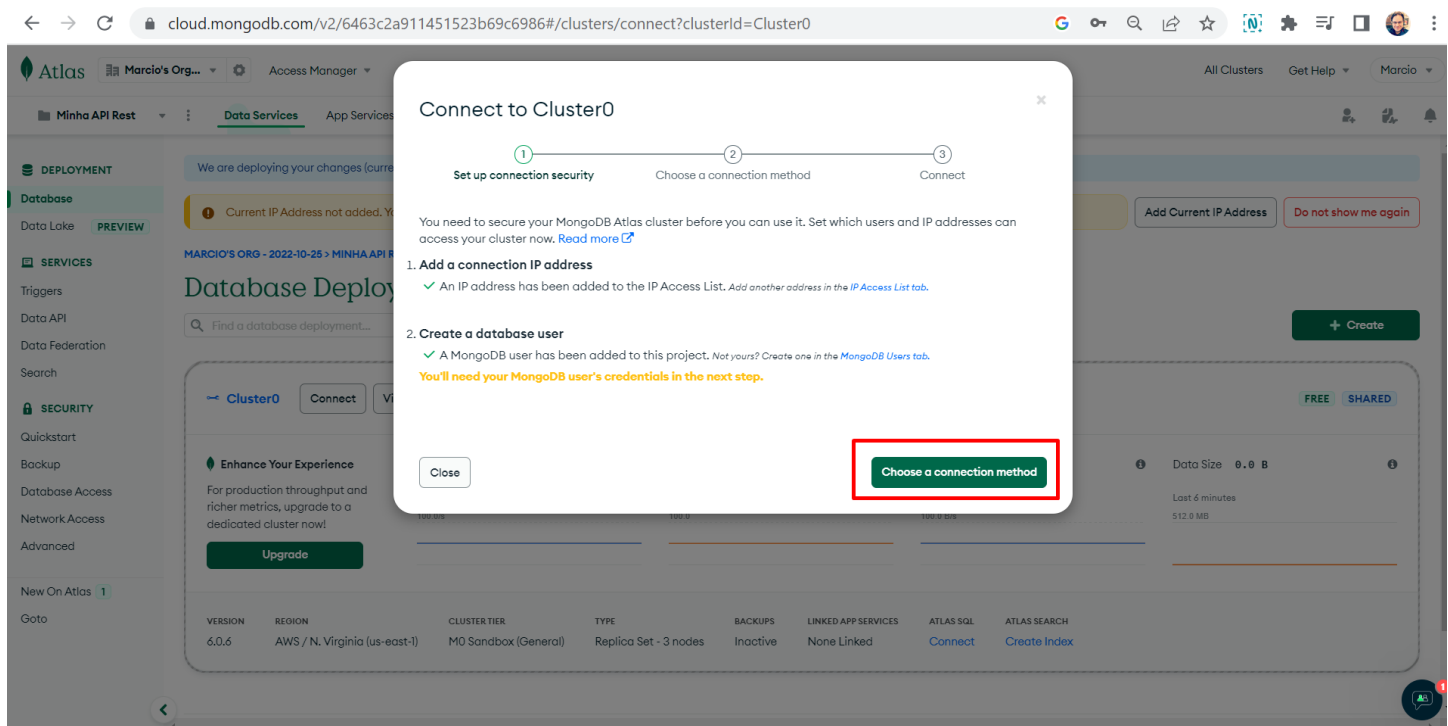
11 – Clique em Create Database User.



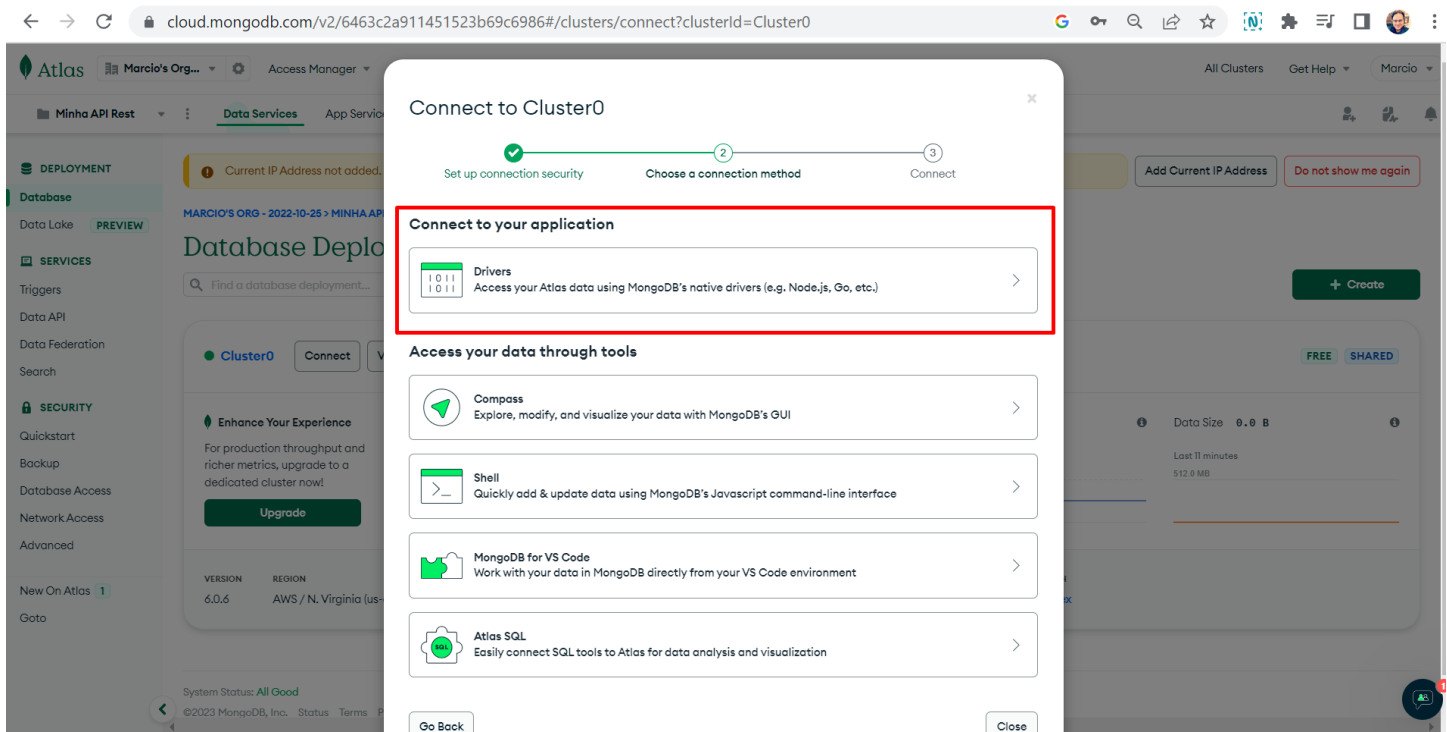
12 – Clique também em Add IP Address



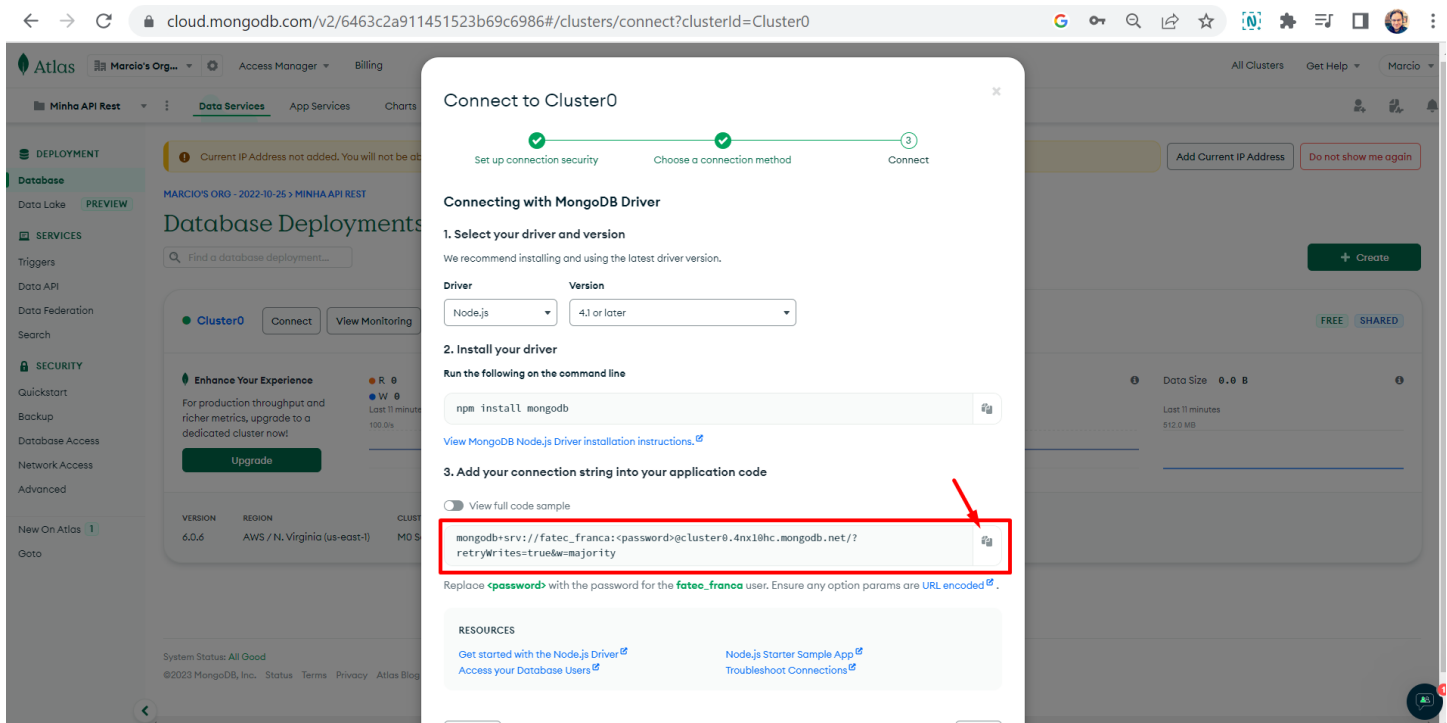
13 – Após adicionar o IP e Usuário/Senha, clique em Choose a connection method.



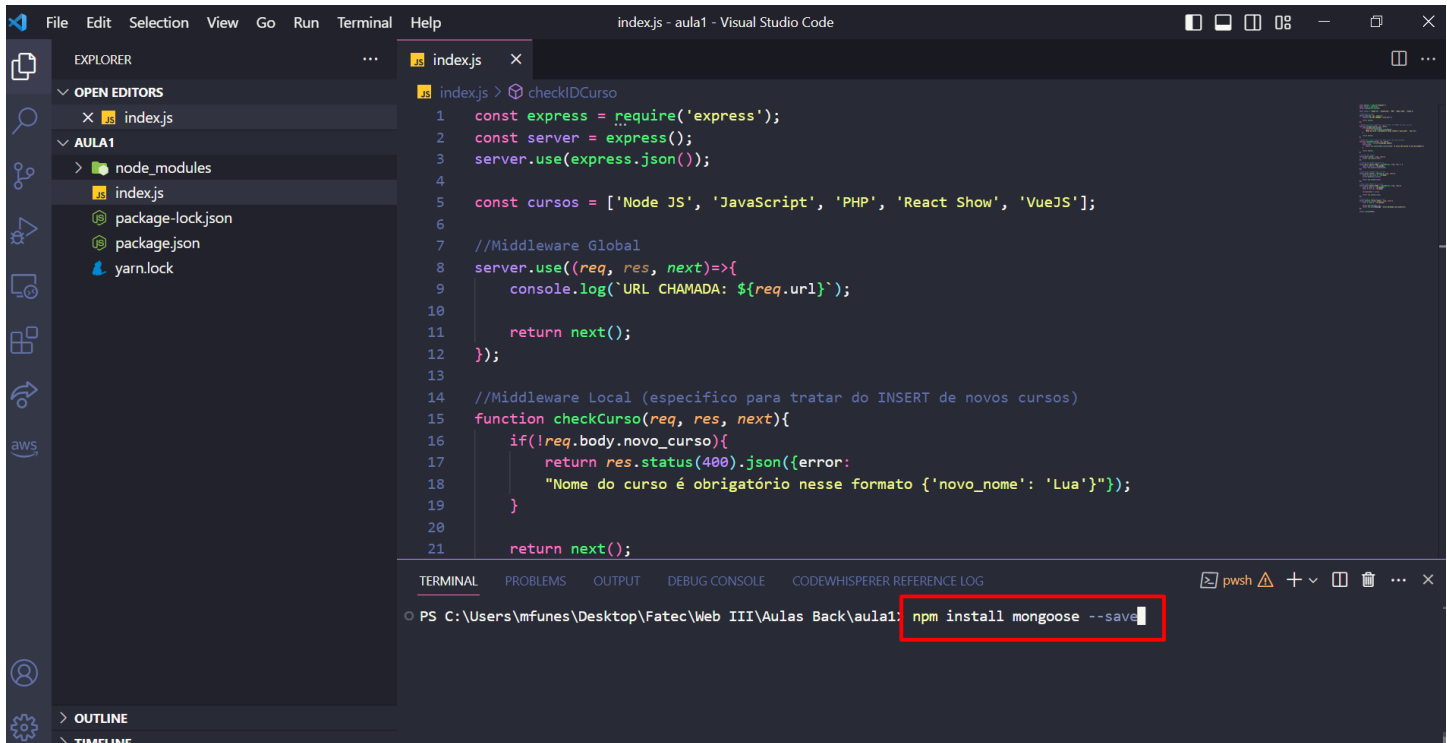
14 – Vamos escolher o método Drives pois ele permite que uma aplicação externa faça acesso ao Banco de Dados.



15 – Aqui é possível a linguagem que nossa aplicação utiliza, vamos manter Node.js, a versão. O que queremos aqui é a string de conexão, copie a string abaixo e salve junto com seu usuário e senha.



16 – Projeto no Atlas criado, vamos agora voltar a nossa aplicação. Continuando onde paramos na última aula, agora precisamos instalar o Drive de conexão chamado mongoose. Digite no terminal “npm install mongoose –save” e dê enter. Mais informações sobre o Mongoose segue o link: <https://mongoosejs.com/>

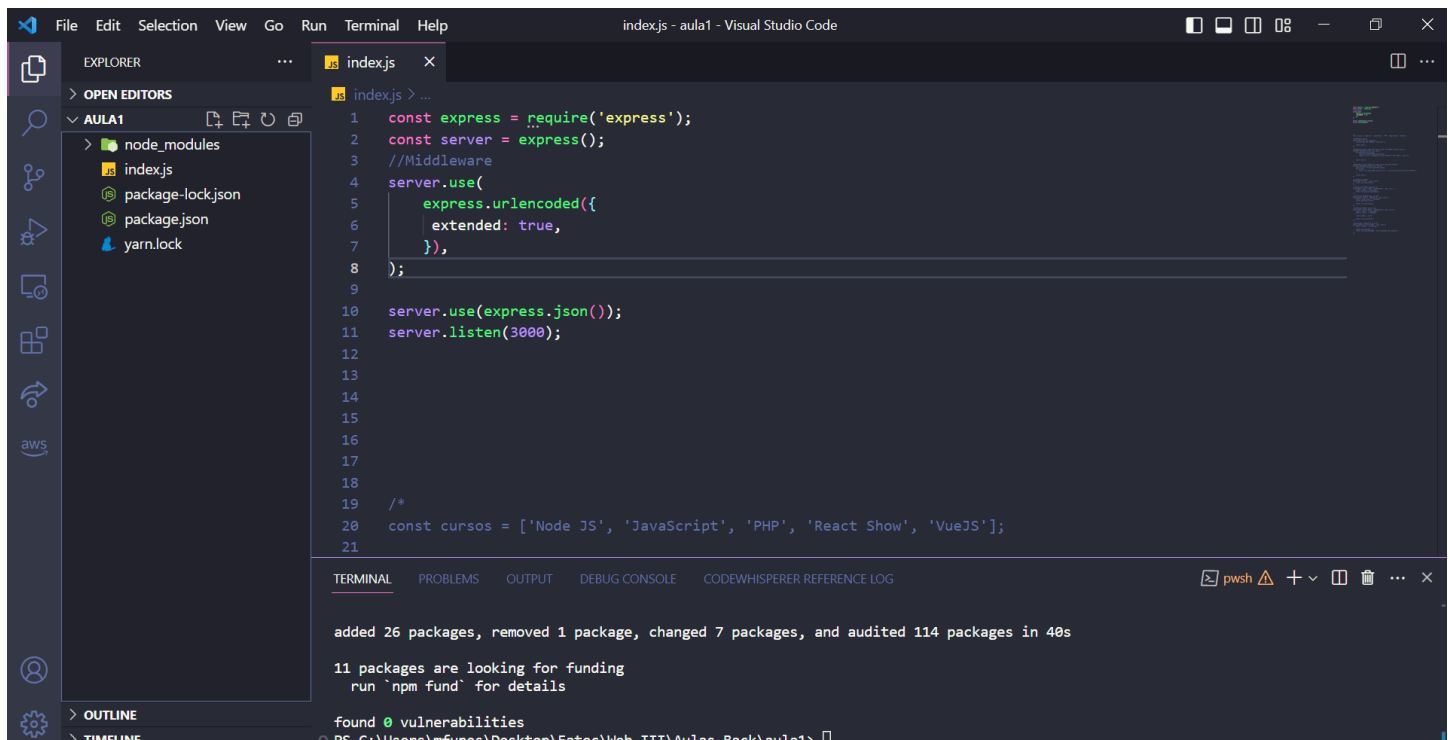


The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays the project structure with files like index.js, package-lock.json, package.json, and yarn.lock. The main editor area shows the content of index.js, which includes code for setting up an Express server with a global middleware and a local checkCurso function. The Terminal panel at the bottom shows the command prompt with the command `npm install mongoose --save` entered, which is highlighted by a red rectangle.

```
1  const express = require('express');
2  const server = express();
3  server.use(express.json());
4
5  const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
6
7  //Middleware Global
8  server.use((req, res, next)=>{
9    console.log(`URL CHAMADA: ${req.url}`);
10
11    return next();
12  });
13
14  //Middleware Local (especifico para tratar do INSERT de novos cursos)
15  function checkCurso(req, res, next){
16    if(!req.body.novo_curso){
17      return res.status(400).json({error:
18        "Nome do curso é obrigatório nesse formato {'novo_nome': 'Lua'}"});
19    }
20
21    return next();
```

PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1> npm install mongoose --save

17 – Vamos fazer a configuração começando do zero, caso queira comente o conteúdo da aula passada para futuras consultas. Vamos criar um middleware que permita a manipulação de url. Deixe o código como abaixo:



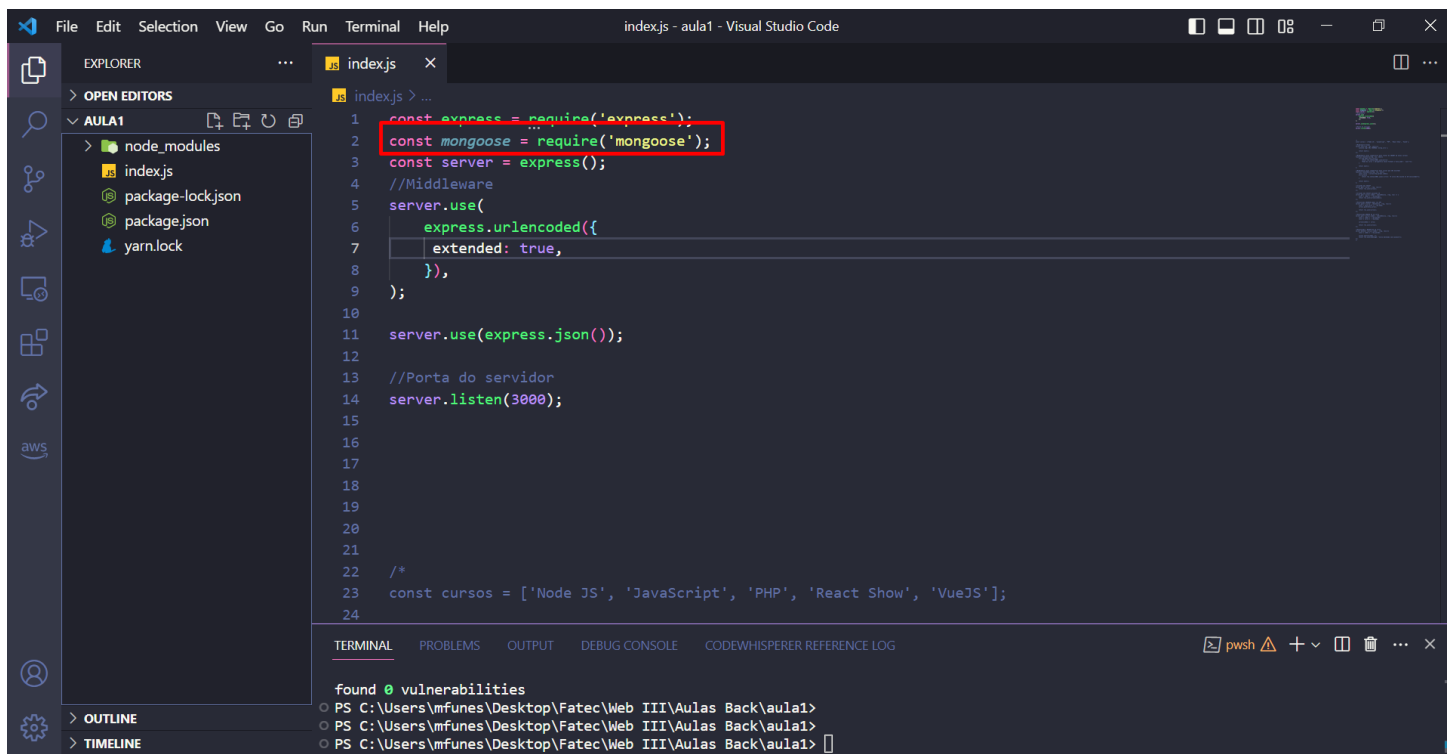
The screenshot shows the Visual Studio Code interface with the file explorer on the left displaying the project structure: AULA1, node_modules, index.js, package-lock.json, package.json, and yarn.lock. The main editor window shows the index.js file with the following code:

```
1 const express = require('express');
2 const server = express();
3 //Middleware
4 server.use(
5   express.urlencoded({
6     extended: true,
7   }),
8 );
9
10 server.use(express.json());
11 server.listen(3000);
12
13
14
15
16
17
18
19 /*
20 const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
21
```

The terminal at the bottom shows the output of the command `npm install`:

```
added 26 packages, removed 1 package, changed 7 packages, and audited 114 packages in 40s
11 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
```

18 – Vamos importar o mongoose para o projeto, como feito abaixo:



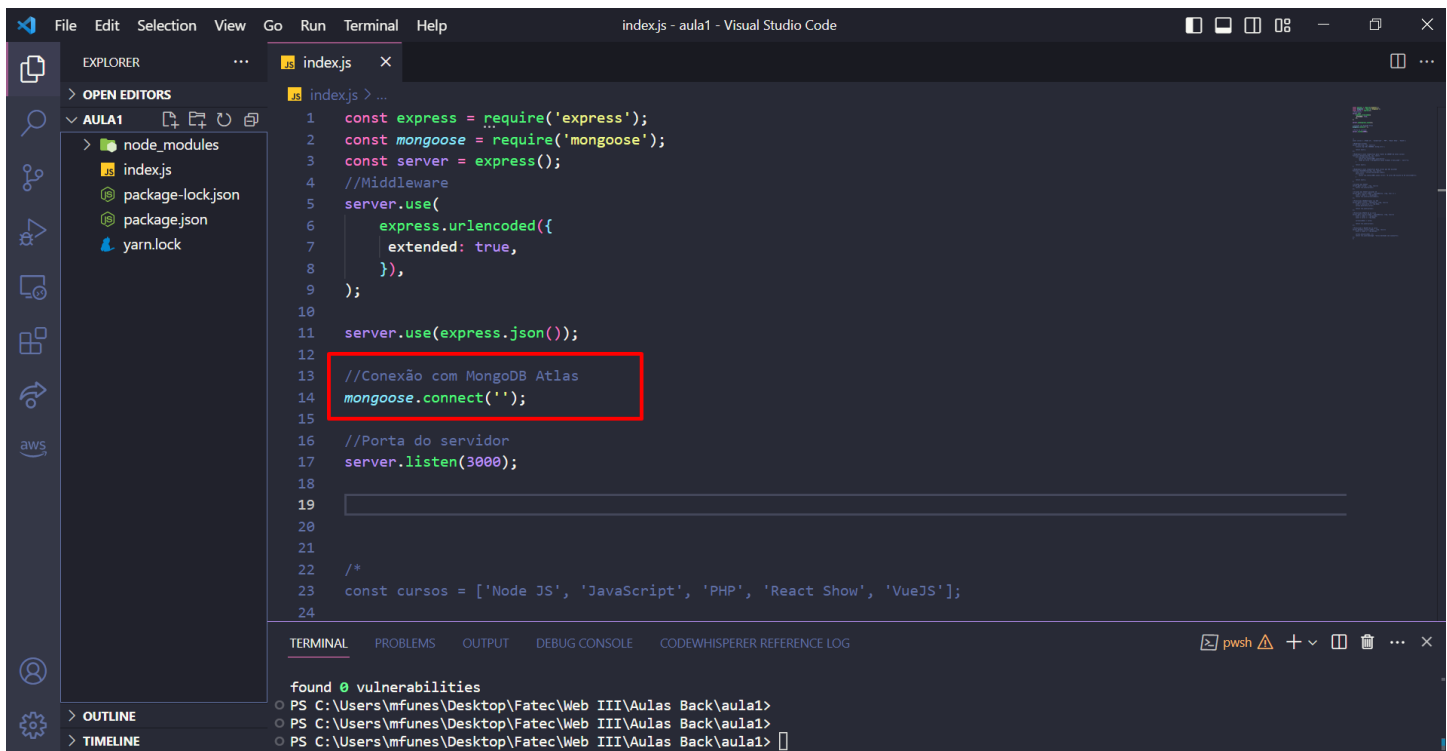
The screenshot shows the Visual Studio Code interface with the file explorer on the left displaying the project structure: AULA1, node_modules, index.js, package-lock.json, package.json, and yarn.lock. The main editor window shows the index.js file with the following code:

```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const server = express();
4 //Middleware
5 server.use(
6   express.urlencoded({
7     extended: true,
8   }),
9 );
10
11 server.use(express.json());
12
13 //Porta do servidor
14 server.listen(3000);
15
16
17
18
19
20
21
22 /*
23 const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
24
```

The terminal at the bottom shows the output of the command `npm install`:

```
found 0 vulnerabilities
PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>
PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>
PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>
```

19 – Com o mongoose importado ao projeto, podemos utilizar uma de suas funções chamada connect, como baixo:



```
File Edit Selection View Go Run Terminal Help
index.js - aula1 - Visual Studio Code

EXPLORER
> OPEN EDITORS
  AULA1
    node_modules
    index.js
    package-lock.json
    package.json
    yarn.lock

index.js
1  const express = require('express');
2  const mongoose = require('mongoose');
3  const server = express();
4  //Middleware
5  server.use(
6    express.urlencoded({
7      extended: true,
8    }),
9  );
10
11 server.use(express.json());
12
13 //Conexão com MongoDB Atlas
14 mongoose.connect('');
15
16 //Porta do servidor
17 server.listen(3000);
18
19
20
21 /*
22
23 const cursos = ['Node JS', 'JavaScript', 'PHP', 'React Show', 'VueJS'];
24

```

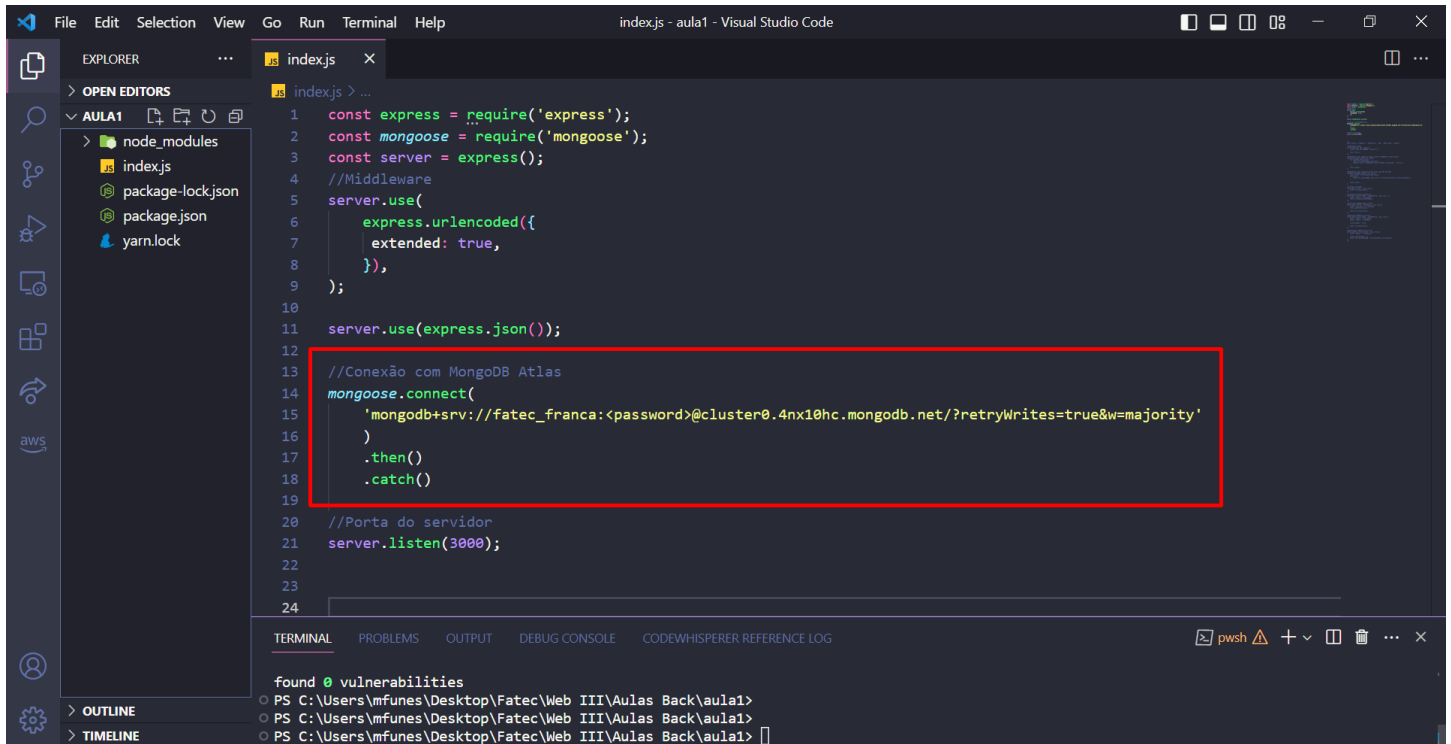
found 0 vulnerabilities

PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>

PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>

PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>

20 – Cole a string de conexão que copiamos do Atlas dentro da função. Como uma conexão pode falar, vamos usar aqui o then e catch para verificar o status da conexão.



```
1  const express = require('express');
2  const mongoose = require('mongoose');
3  const server = express();
4  //Middleware
5  server.use(
6    express.urlencoded({
7      extended: true,
8    }),
9  );
10
11 server.use(express.json());
12
13 //Conexão com MongoDB Atlas
14 mongoose.connect(
15   'mongodb+srv://fatec_franca:<password>@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority'
16 )
17 .then()
18 .catch()
19
20 //Porta do servidor
21 server.listen(3000);
22
23
24
```

found 0 vulnerabilities

PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>

PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>

PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>

21 – Vamos agora configurar na aplicação nosso usuário e senha. Crie duas variáveis, coloque o usuário e senha em cada uma. Mude as aspas simples para template string ``.

```
5 server.use(
6   express.urlencoded({
7     extended: true,
8   }),
9 );
10
11 server.use(express.json());
12
13 //Conexão com MongoDB Atlas
14 const DB_USER = 'fatec_franca';
15 const DB_PASSWORD = encodeURIComponent('Z0YgQexwQ6kHI56t');
16
17 mongoose.connect(
18   `mongodb+srv://${DB_USER}:${DB_PASSWORD}@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority`
19 )
20 .then()
21 .catch()
22
23 //Porta do servidor
24 server.listen(3000);
25
26
27
28
```

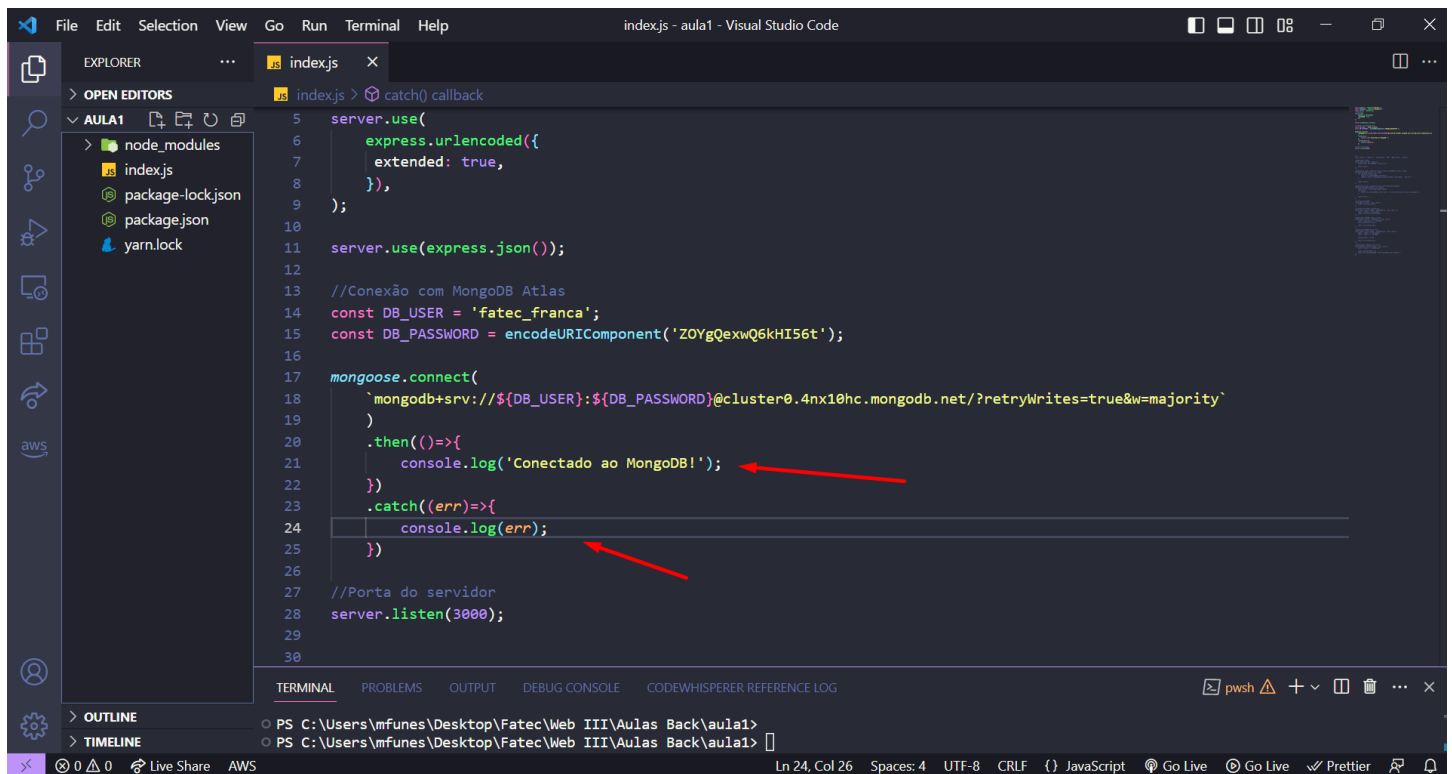
found 0 vulnerabilities

PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>

PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>

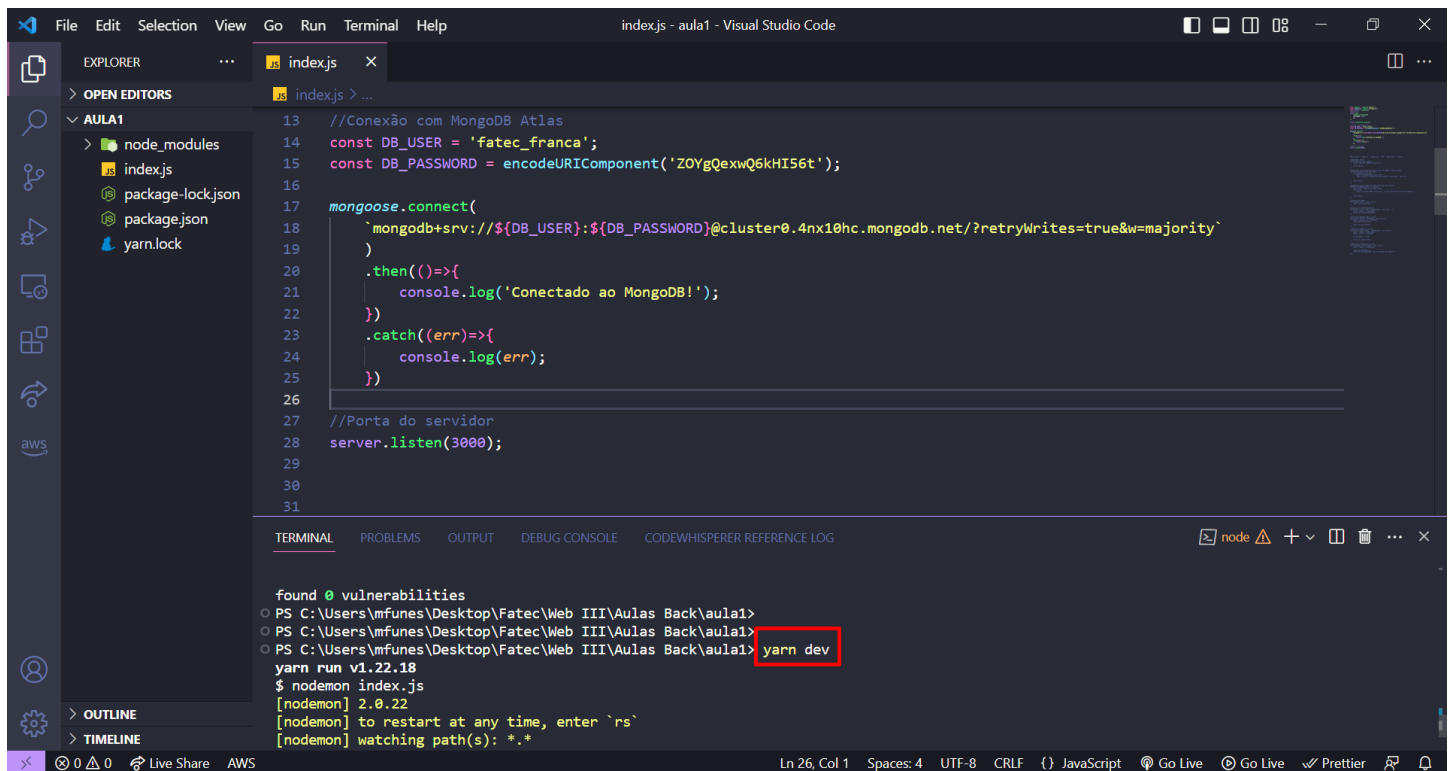
PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>

22 – Agora no then e catch vamos verificar se a conexão aconteceu e se houve erro qual o tipo de erro para assim podermos tratar.



```
5 server.use(
6   express.urlencoded({
7     extended: true,
8   }),
9 );
10
11 server.use(express.json());
12
13 //Conexão com MongoDB Atlas
14 const DB_USER = 'fatec_franca';
15 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
16
17 mongoose.connect(
18   `mongodb+srv://${DB_USER}:${DB_PASSWORD}@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority`
19 )
20 .then(()=>{
21   console.log('Conectado ao MongoDB!');
22 })
23 .catch((err)=>{
24   console.log(err);
25 })
26
27 //Porta do servidor
28 server.listen(3000);
29
30
```

23 – Bora testar a conexão, suba nossa aplicação utilizando o yarn dev.

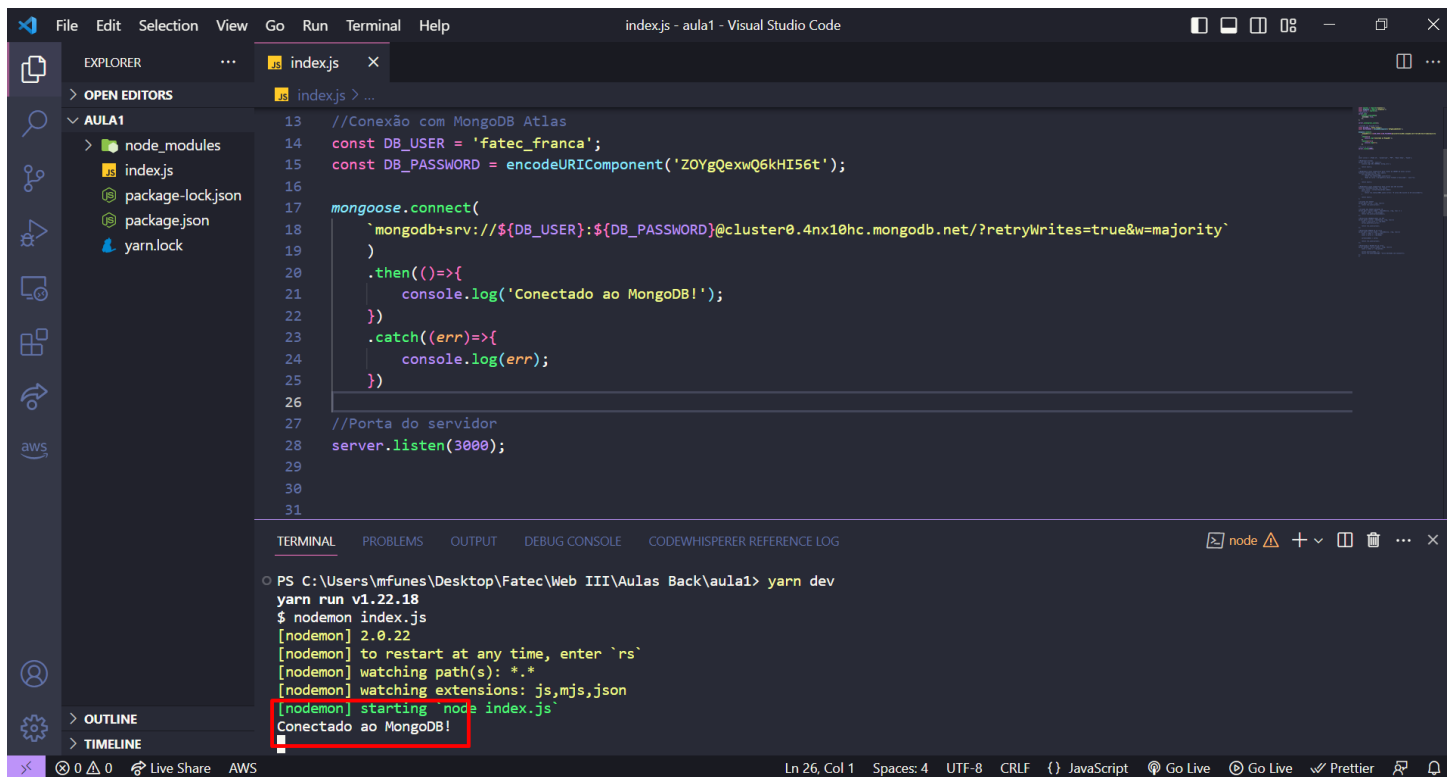


```
13 //Conexão com MongoDB Atlas
14 const DB_USER = 'fatec_franca';
15 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
16
17 mongoose.connect(
18   `mongodb+srv://${DB_USER}:${DB_PASSWORD}@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority`
19 )
20 .then(()=>{
21   console.log('Conectado ao MongoDB!');
22 })
23 .catch((err)=>{
24   console.log(err);
25 })
26
27 //Porta do servidor
28 server.listen(3000);
29
30
31
```

```
found 0 vulnerabilities
PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>
PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1>
PS C:\Users\mfunes\Desktop\Fatec\Web III\Aulas Back\aula1> yarn dev
yarn run v1.22.18
$ nodemon index.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*

```

24 – Após alguns segundo podemos ver que nossa conexão foi um sucesso :D

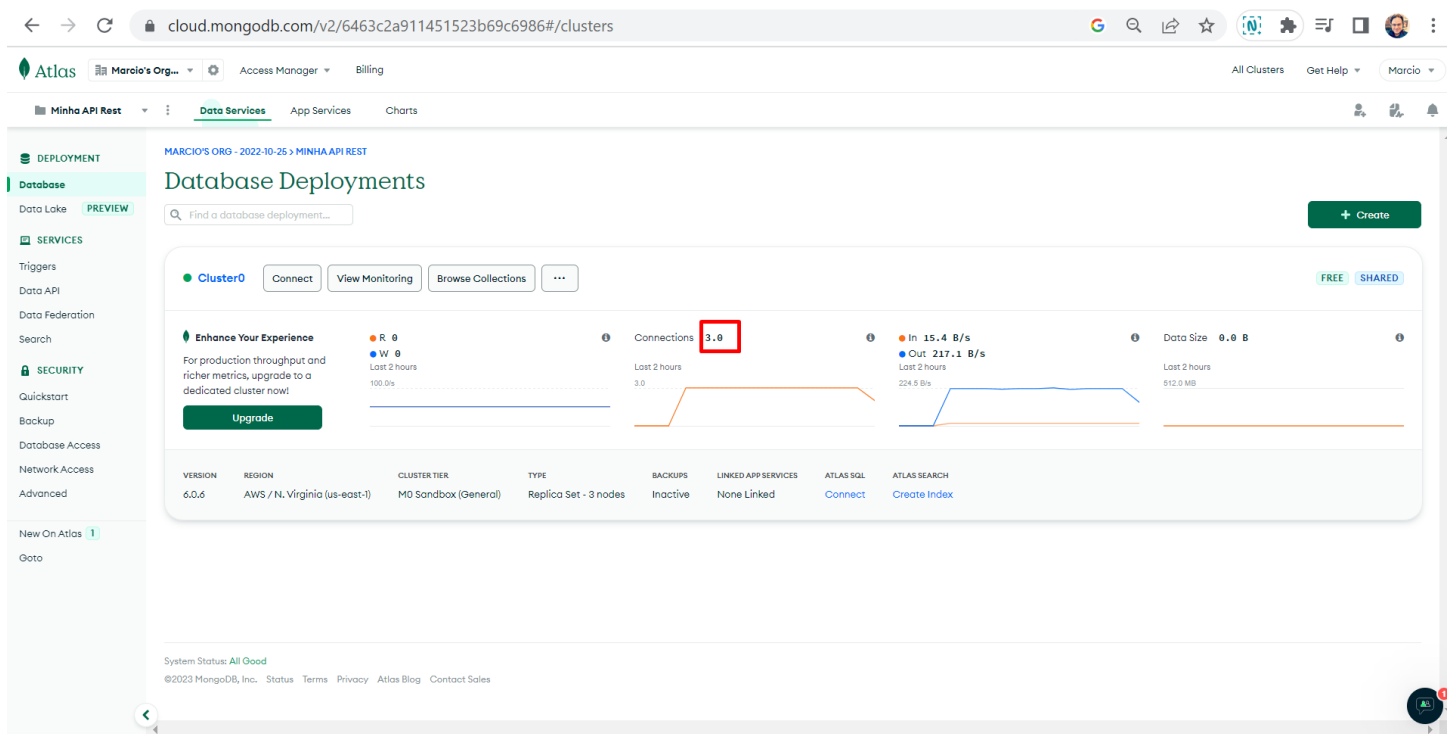


The screenshot shows the Visual Studio Code editor with a file named `index.js` open. The code in the file is as follows:

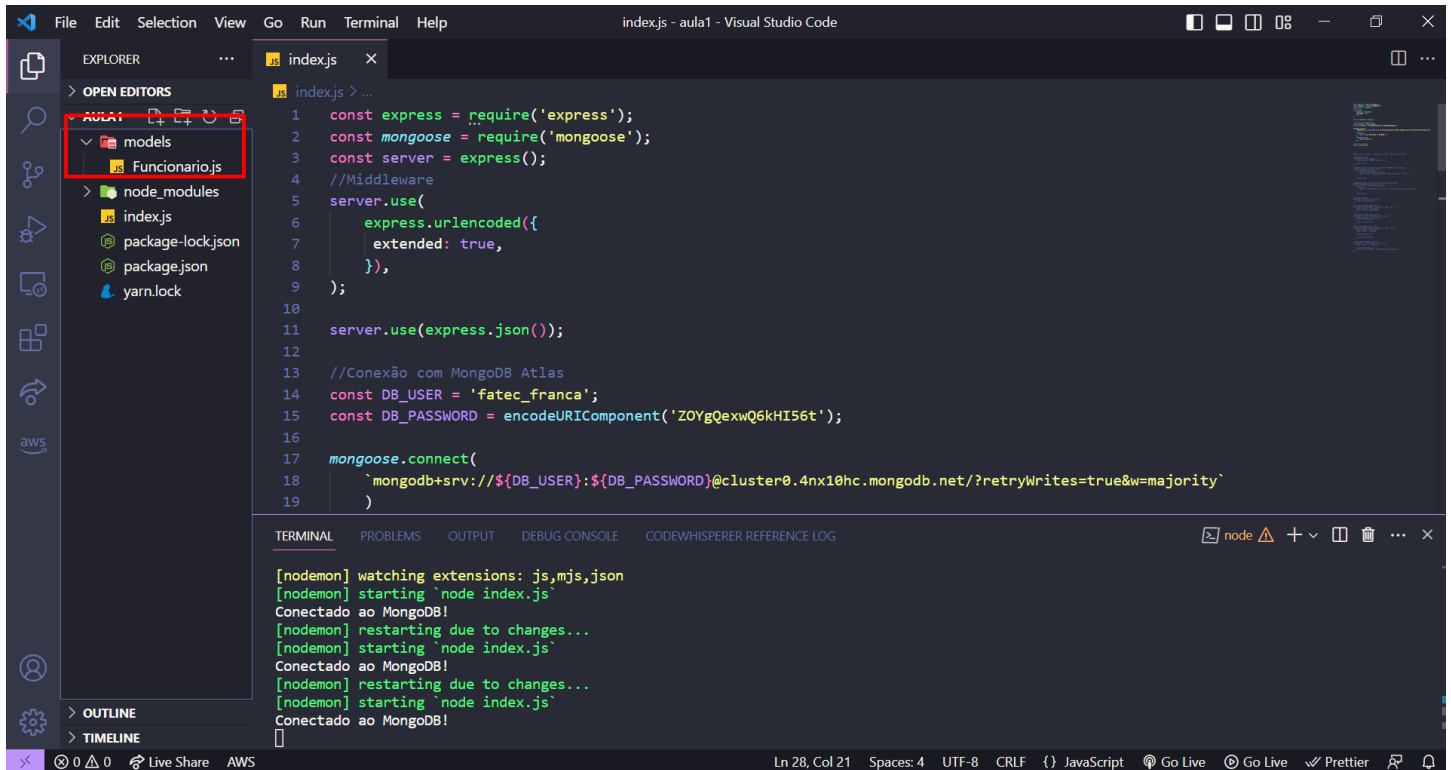
```
13 //Conexão com MongoDB Atlas
14 const DB_USER = 'fatec_franca';
15 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
16
17 mongoose.connect(
18   `mongodb+srv://${DB_USER}:${DB_PASSWORD}@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority`
19 )
20 .then(()=>{
21   console.log('Conectado ao MongoDB!');
22 })
23 .catch((err)=>{
24   console.log(err);
25 })
26
27 //Porta do servidor
28 server.listen(3000);
29
30
31
```

The terminal at the bottom shows the command `yarn dev` being executed, which runs `nodemon index.js`. The output shows that the application is running and has successfully connected to MongoDB Atlas, as indicated by the message `Conectado ao MongoDB!` which is highlighted with a red box in the original image.

25 – Na interface do Atlas podemos ver que existem 3 conexões ativas (2 são internas do Atlas e a 3ª é nossa aplicação).



26 – Agora que temos a conexão realizada, precisamos modelar o objeto que iremos enviar ao MongoDB. Esse é o conceito de **ODM – Object Data Modeling** (Modelagem de Dados de Objeto). Em nosso exemplo o objeto será um funcionário. Crie uma pasta chamada models e dentro dela Funcionario.js.



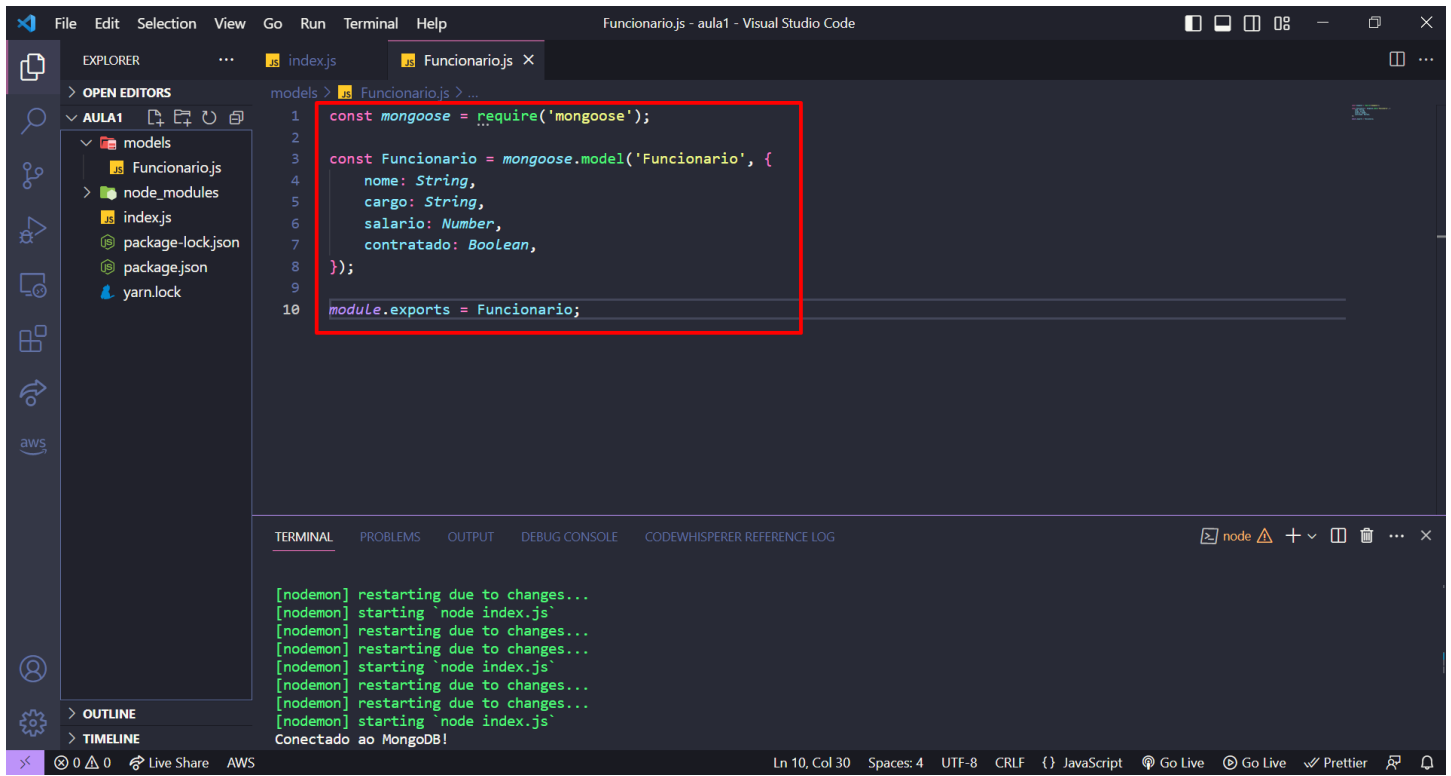
The screenshot shows the Visual Studio Code interface with the following components:

- EXPLORER:** The file explorer on the left shows a project structure with a folder named `models` containing a file `Funcionario.js`. A red box highlights the `models` folder and `Funcionario.js` file.
- EDITOR:** The main editor window shows the `index.js` file. The code includes imports for `express` and `mongoose`, server setup, and a connection to MongoDB Atlas. The code is as follows:

```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const server = express();
4 //Middleware
5 server.use(
6   express.urlencoded({
7     extended: true,
8   }),
9 );
10
11 server.use(express.json());
12
13 //Conexão com MongoDB Atlas
14 const DB_USER = 'fatec_franca';
15 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
16
17 mongoose.connect(
18   `mongodb+srv://${DB_USER}:${DB_PASSWORD}@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority`
19 )
```
- TERMINAL:** The terminal at the bottom shows the output of running the application. It indicates that the application is watching for changes, starting `node index.js`, and connecting to MongoDB. The output is as follows:

```
[nodeemon] watching extensions: js,mjs,json
[nodeemon] starting `node index.js`
Conectado ao MongoDB!
[nodeemon] restarting due to changes...
[nodeemon] starting `node index.js`
Conectado ao MongoDB!
[nodeemon] restarting due to changes...
[nodeemon] starting `node index.js`
Conectado ao MongoDB!
```

27 – Aqui vamos definir o modelo ao qual o MongoDB deve considerar ao persistir os dados no banco. Nosso funcionário terá um nome, cargo, salario e um campo chamado desligado para saber se ele é funcionário atual ou foi demitido.

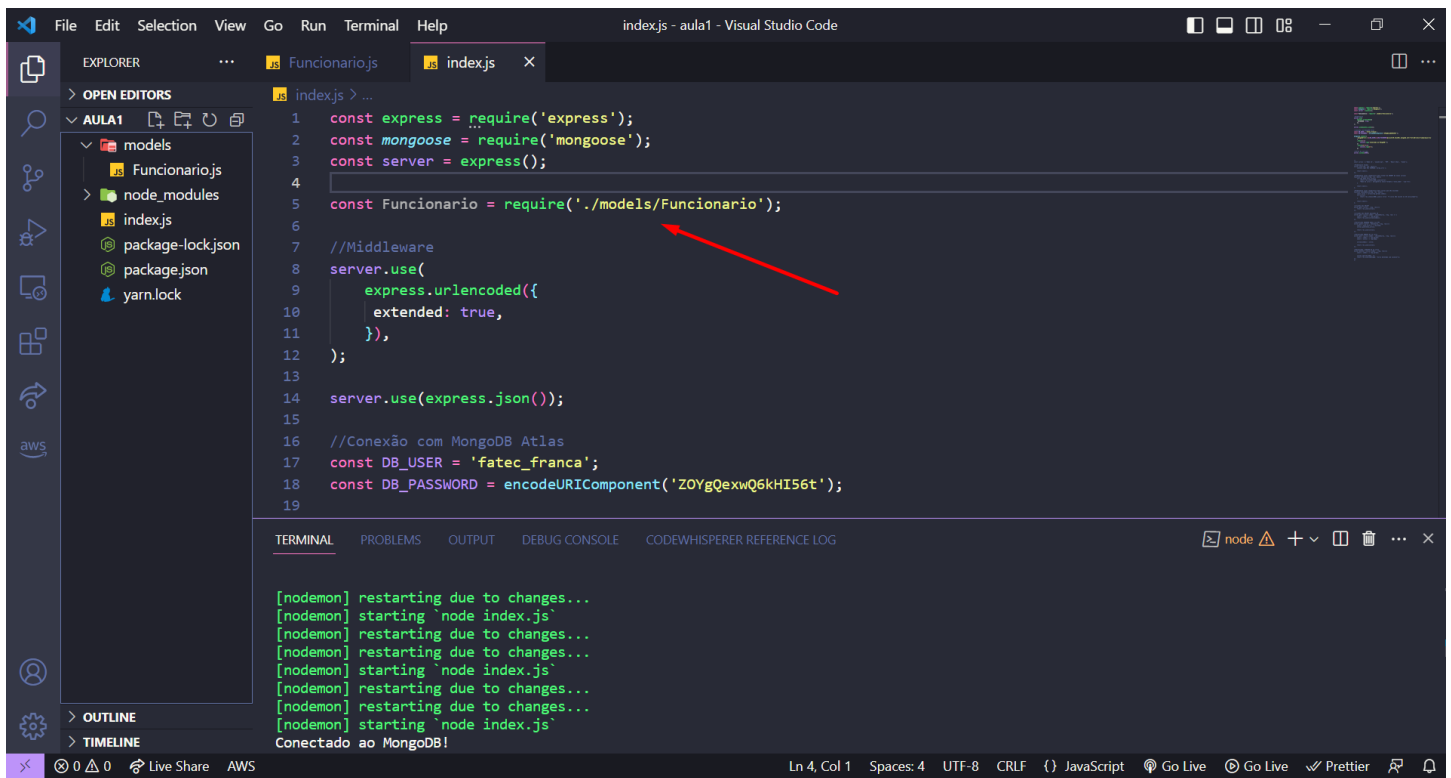


The screenshot shows the Visual Studio Code interface with the file 'Funcionario.js' open in the editor. The code defines a Mongoose model for 'Funcionario' with the following fields: 'nome' (String), 'cargo' (String), 'salario' (Number), and 'contratado' (Boolean). The code is as follows:

```
1 const mongoose = require('mongoose');
2
3 const Funcionario = mongoose.model('Funcionario', {
4   nome: String,
5   cargo: String,
6   salario: Number,
7   contratado: Boolean,
8 });
9
10 module.exports = Funcionario;
```

The Explorer sidebar on the left shows the project structure with files like 'index.js', 'package-lock.json', 'package.json', and 'yarn.lock'. The Terminal at the bottom shows logs from nodemon, including multiple restarts due to changes and a message 'Conectado ao MongoDB!'.

28 – Voltando a nossa aplicação, vamos importar o modelo para poder acessar as funções internas dele.



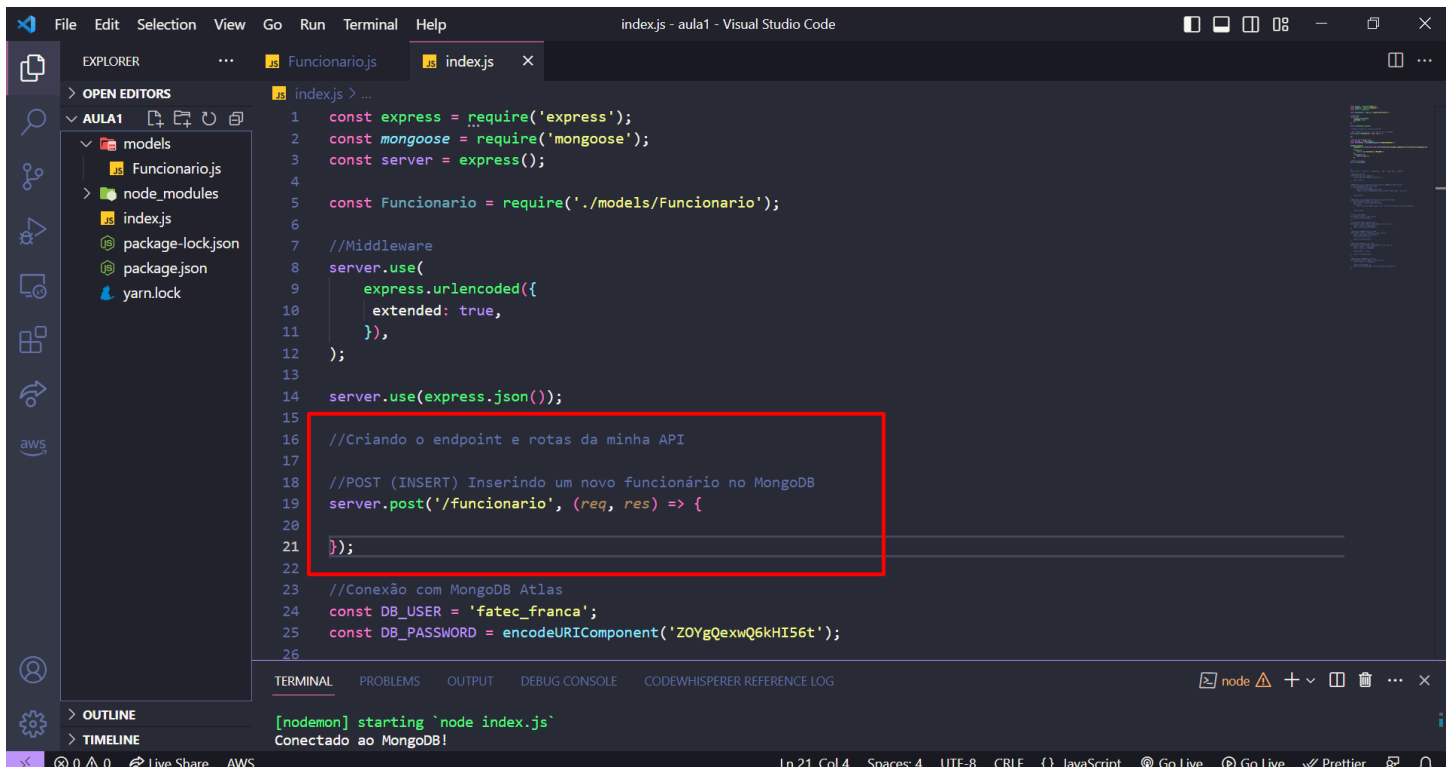
The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left shows the project structure with a folder named 'AULA1' containing subfolders 'models' and 'node_modules'. The 'models' folder is expanded, showing 'Funcionario.js'. The main editor area displays the 'index.js' file. A red arrow points to the line `const Funcionario = require('./models/Funcionario');` on line 5. The terminal at the bottom shows the output of running the application, indicating that the application is restarting due to changes and successfully connecting to MongoDB.

```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const server = express();
4
5 const Funcionario = require('./models/Funcionario');
6
7 //Middleware
8 server.use(
9   express.urlencoded({
10     extended: true,
11   }),
12 );
13
14 server.use(express.json());
15
16 //Conexão com MongoDB Atlas
17 const DB_USER = 'fatec_franca';
18 const DB_PASSWORD = encodeURIComponent('Z0YgQexwQ6kHI56t');
19
```

Terminal Output:

```
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Conectado ao MongoDB!
```

29 – Agora temos tudo para começar a criar as rotas da nossa API. Como nosso banco está vazio, vamos começar o CRUD permitindo a Inserção de dados por meio de POST. Crie uma rota ‘/funcionario’ como abaixo:



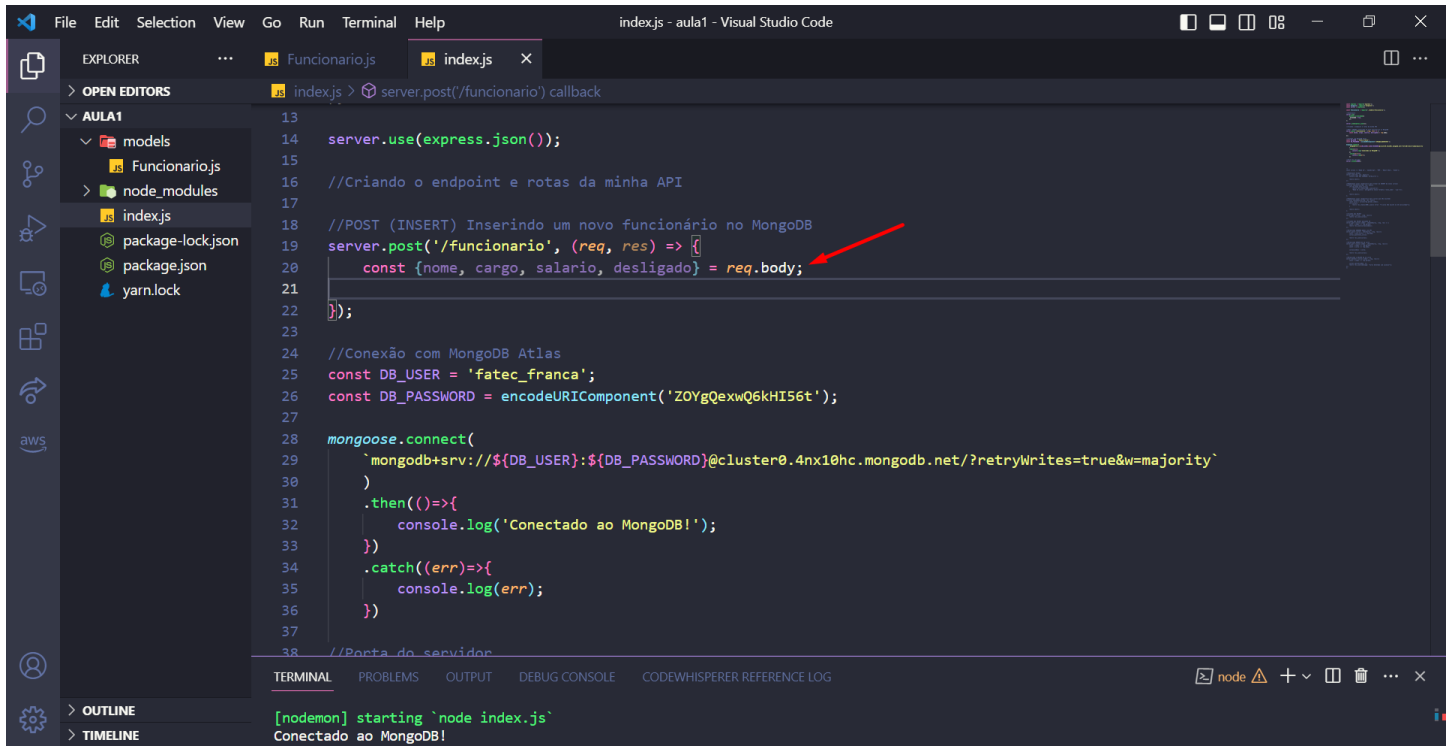
```
1  const express = require('express');
2  const mongoose = require('mongoose');
3  const server = express();
4
5  const Funcionario = require('./models/Funcionario');
6
7  //Middleware
8  server.use(
9    express.urlencoded({
10      extended: true,
11    }),
12  );
13
14  server.use(express.json());
15
16  //Criando o endpoint e rotas da minha API
17
18  //POST (INSERT) Inserindo um novo funcionario no MongoDB
19  server.post('/funcionario', (req, res) => {
20
21  });
22
23  //Conexão com MongoDB Atlas
24  const DB_USER = 'fatec_franca';
25  const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
26
```

Terminal output:

```
[nodeemon] starting `node index.js`
Conectado ao MongoDB!
```

30 – Sabemos que o POST sempre espera alguma coisa dentro do body de requisição quando um cliente o utiliza via API. Então vamos definir uma constante que diga qual o formato que esperamos que esse JSON chegue via request body. Isso se chama Destructuring (Desestruturação) de um objeto que o JavaScript moderno permite (mais informações:

<https://programandosolucoes.dev.br/2022/09/06/destructuring-javascript/>)

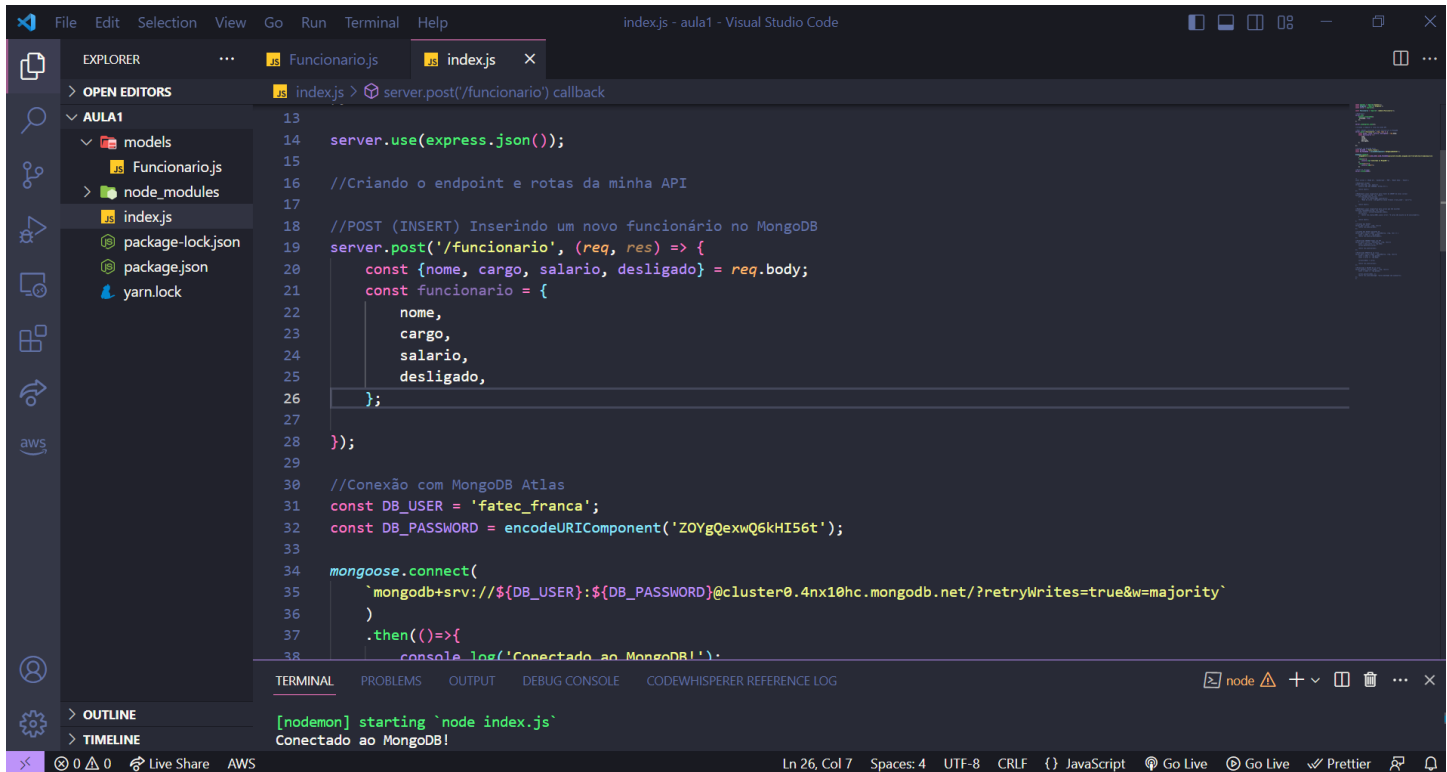


```
13
14 server.use(express.json());
15
16 //Criando o endpoint e rotas da minha API
17
18 //POST (INSERT) Inserindo um novo funcionário no MongoDB
19 server.post('/funcionario', (req, res) => {
20   const {nome, cargo, salario, desligado} = req.body;
21 });
22
23
24 //Conexão com MongoDB Atlas
25 const DB_USER = 'fatec_franca';
26 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
27
28 mongoose.connect(
29   `mongodb+srv://${DB_USER}:${DB_PASSWORD}@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority`
30 )
31 .then(()=>{
32   console.log('Conectado ao MongoDB!');
33 })
34 .catch((err)=>{
35   console.log(err);
36 })
37
38 //Porta do servidor
```

TERMINAL

```
[nodemon] starting `node index.js`
Conectado ao MongoDB!
```

31 – Vamos agora criar o objeto chamado “funcionario” isso facilita nosso trabalho pois iremos passar esse objeto para o banco de dados. Antes criamos o modelo funcionário, e agora baseado nesse modelo estamos criando um objeto, seguindo a lógica de um ODM.

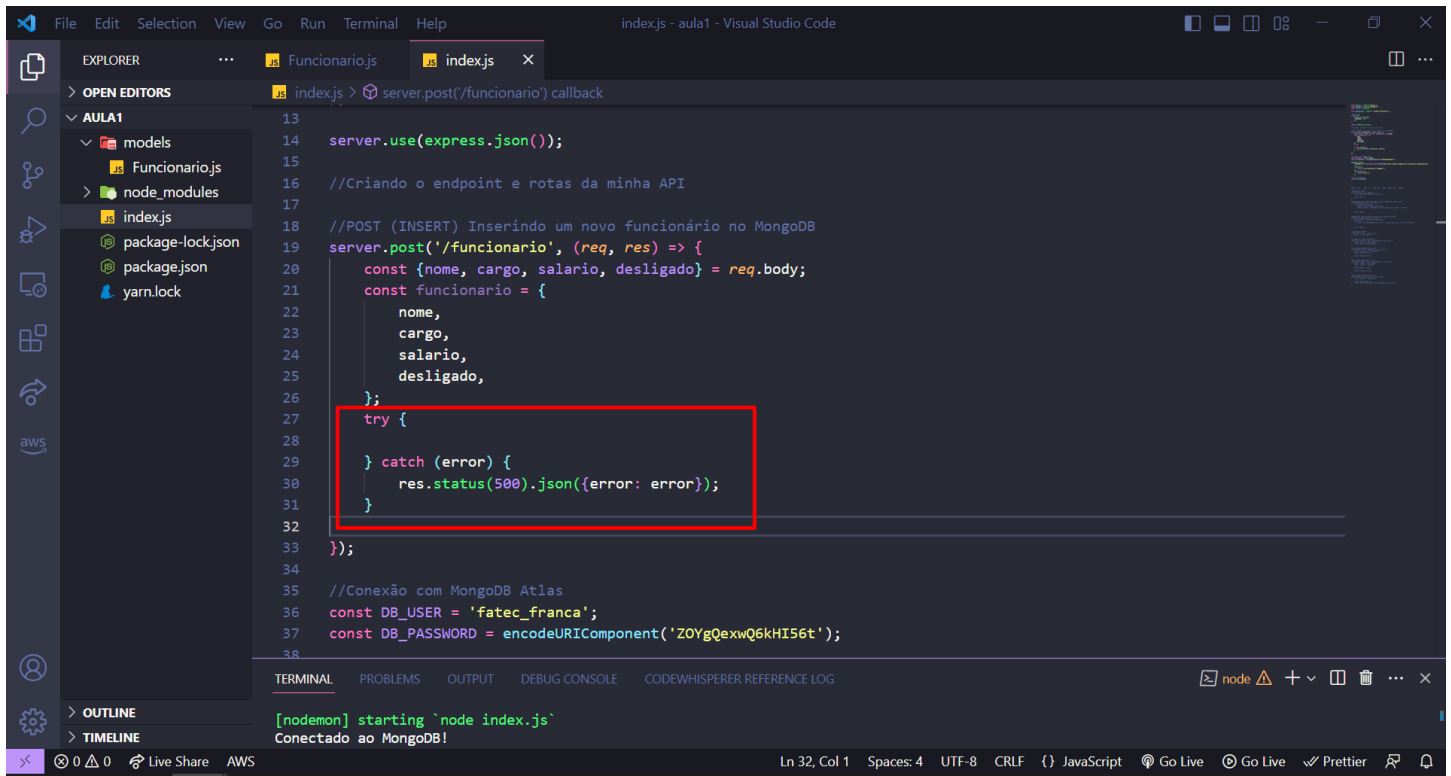


```
13
14 server.use(express.json());
15
16 //Criando o endpoint e rotas da minha API
17
18 //POST (INSERT) Inserindo um novo funcionário no MongoDB
19 server.post('/funcionario', (req, res) => {
20   const {nome, cargo, salario, desligado} = req.body;
21   const funcionario = {
22     nome,
23     cargo,
24     salario,
25     desligado,
26   };
27
28 });
29
30 //Conexão com MongoDB Atlas
31 const DB_USER = 'fatec_franca';
32 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
33
34 mongoose.connect(
35   `mongodb+srv://${DB_USER}:${DB_PASSWORD}@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority`
36 )
37 .then(()=>{
38   console.log('Conectado ao MongoDB!');
39 });
```

TERMINAL

```
[nodemon] starting `node index.js`
Conectado ao MongoDB!
```

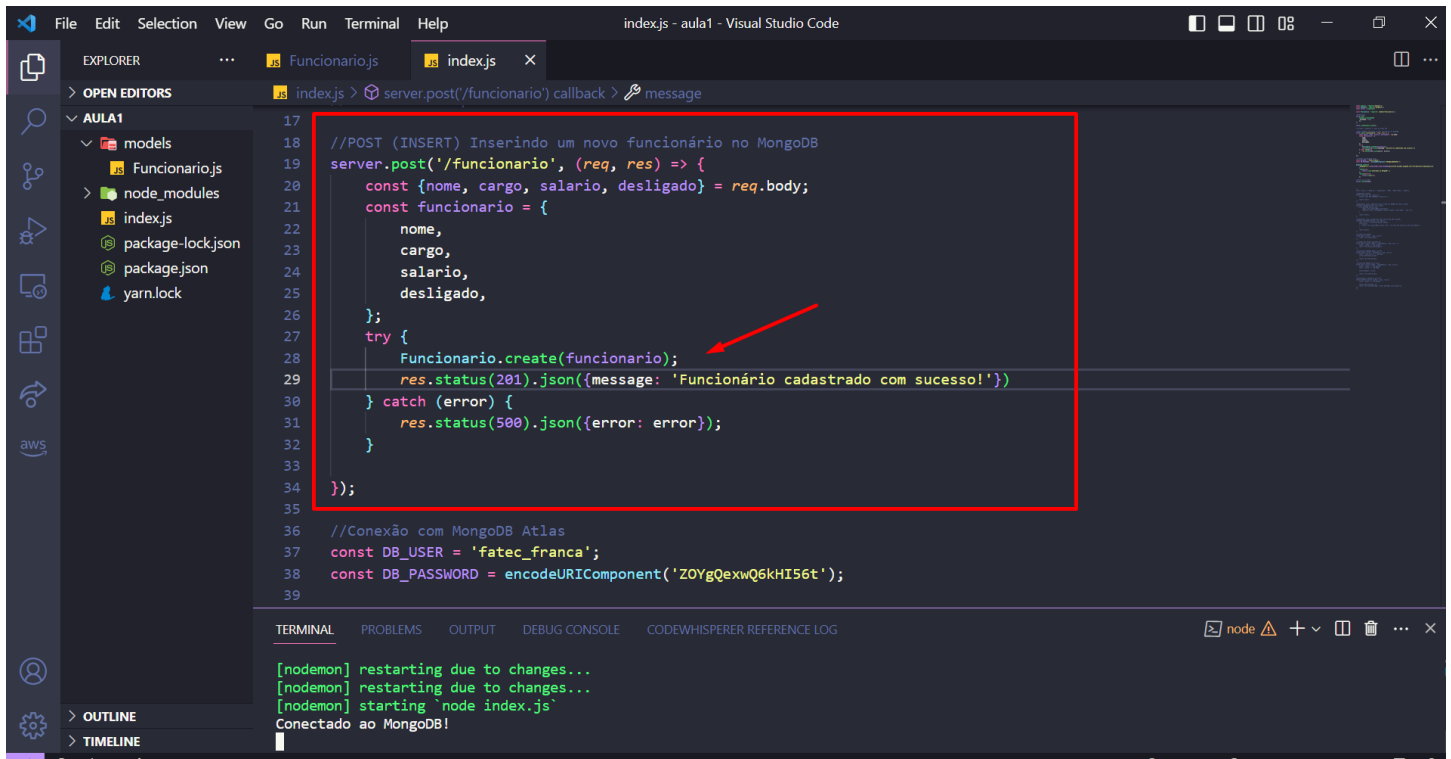

32 – Vamos fazer um try catch, onde, no catch verificamos se existe algum erro relacionado a conexão com o banco. Ou seja, se algum cliente tentar usar nossa API e não conseguir utilizar esse POST, ele receberá uma mensagem de erro.



```
13 server.use(express.json());
14
15 //Criando o endpoint e rotas da minha API
16
17 //POST (INSERT) Inserindo um novo funcionário no MongoDB
18 server.post('/funcionario', (req, res) => {
19   const {nome, cargo, salario, desligado} = req.body;
20   const funcionario = {
21     nome,
22     cargo,
23     salario,
24     desligado,
25   };
26   try {
27   } catch (error) {
28     res.status(500).json({error: error});
29   }
30 });
31
32 //Conexão com MongoDB Atlas
33 const DB_USER = 'fatec_franca';
34 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
```

Ln 32, Col 1 Spaces: 4 UTF-8 CRLF {} JavaScript Go Live Go Live Prettier

33 – E agora de fato, chamamos nosso modelo Funcionario (importado no começo do código) e pela função **create** pedimos para que o MongoDB crie um objeto chamado funcionário contendo a definição de campos feita anteriormente. Vamos também, enviar uma resposta (res) com o código 201 e dizer que o funcionário foi cadastrado com sucesso!



```
17 //POST (INSERT) Inserindo um novo funcionário no MongoDB
18 server.post('/funcionario', (req, res) => {
19   const {nome, cargo, salario, desligado} = req.body;
20   const funcionario = {
21     nome,
22     cargo,
23     salario,
24     desligado,
25   };
26   try {
27     Funcionario.create(funcionario);
28   } catch (error) {
29     res.status(500).json({error: error});
30   }
31 });
32
33
34
35
36 //Conexão com MongoDB Atlas
37 const DB_USER = 'fatec_franca';
38 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
39
```

Terminal output:

```
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Conectado ao MongoDB!
```

34 – Bora testar essa API, usando o Postman vamos criar uma requisição POST para o endereço da nossa API “localhost:3000/funcionario” e enviar. Veja que a API respondeu que o funciário foi criado, porém, onde estão as informações de funcionário?

The screenshot shows the Postman interface with a POST request to `localhost:3000/funcionario`. The response status is `201 Created` with a response time of `72 ms` and a size of `290 B`. The response body is displayed in JSON format, showing a success message.

Request Details:

- Method: POST
- URL: localhost:3000/funcionario

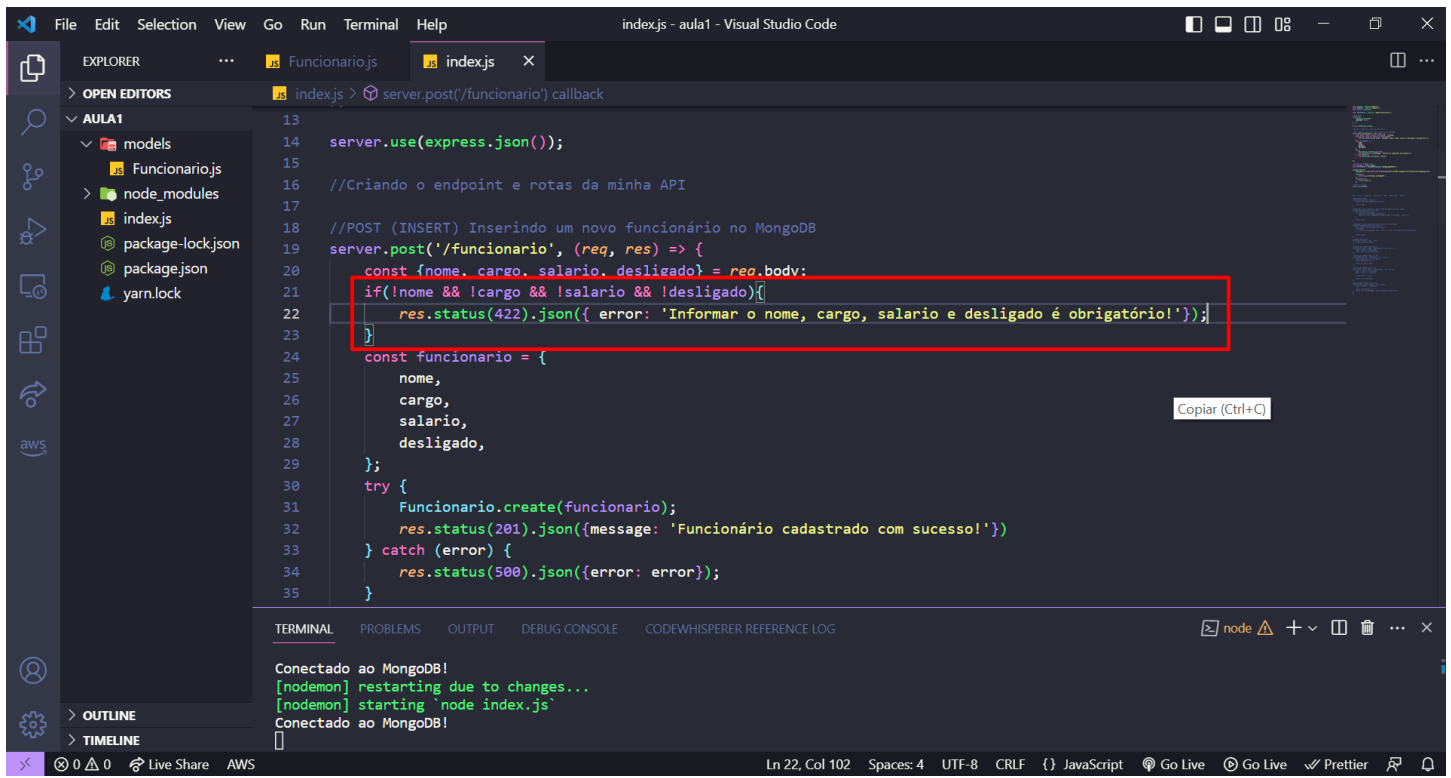
Response Details:

KEY	VALUE	DESCRIPTION
Key	Value	Description

Body (JSON):

```
1 {
2   "message": "Funcionário cadastrado com sucesso!"
3 }
```

35 – Vamos fazer a verificação se os campos que queremos estão contidos dentro do body do POST e caso eles não estejam presente, vamos informar o código 422 e informar o cliente do erro.



```
13
14 server.use(express.json());
15
16 //Criando o endpoint e rotas da minha API
17
18 //POST (INSERT) Inserindo um novo funcionário no MongoDB
19 server.post('/funcionario', (req, res) => {
20   const {nome, cargo, salario, desligado} = req.body;
21   if(!nome && !cargo && !salario && !desligado){
22     res.status(422).json({ error: 'Informar o nome, cargo, salario e desligado é obrigatório!'});
23   }
24   const funcionario = {
25     nome,
26     cargo,
27     salario,
28     desligado,
29   };
30   try {
31     Funcionario.create(funcionario);
32     res.status(201).json({message: 'Funcionário cadastrado com sucesso!'});
33   } catch (error) {
34     res.status(500).json({error: error});
35   }
}
```

Terminal output:

```
Conectado ao MongoDB!
[nodemon] restarting due to changes...
[nodemon] starting 'node index.js'
Conectado ao MongoDB!
```

36 – Veja que agora sim, ele faz o tratamento de erro adequado.

The screenshot shows the Postman interface with a POST request to `localhost:3000/funcionario` under the collection `API NodeJS + MongoDB / Inserindo Funcionarios`. The response is a 422 Unprocessable Entity, which is highlighted with a red box. The response body is displayed in JSON format, showing an error message.

Query Params

KEY	VALUE	DESCRIPTION
Key	Value	Description

Body

422 Unprocessable Entity 43 ms 325 B Save Response

Pretty Raw Preview Visualize JSON

```
1  
2  "error": "Informar o Nome, cargo, salario e desligado é obrigatório!"  
3
```

37 – Vamos agora colocar um Body correto com dados de um funcionário e enviar o POST. Veja se a requisição retornou que o funcionário foi cadastrado!

The screenshot shows the Postman interface with a POST request named "Inserindo Funcionarios" to the endpoint `localhost:3000/funcionario`. The request body is a JSON object with the following data:

```
{
  "nome": "Silvio Santos",
  "cargo": "Apresentador",
  "salario": 1000000,
  "desligado": false
}
```

The response body is a JSON object with a success message:

```
{
  "message": "Funcionário cadastrado com sucesso!"
}
```

Red boxes highlight the request body, the "Body" tab, and the response body. A red arrow points to the "Send" button.

38 – Vamos verificar agora se o dado passou pelo nosso back-end e foi de fato para o banco de dados. Na interface do MongoDB vá em Browser Collections.

The screenshot shows the MongoDB Atlas web interface. The browser address bar indicates the URL: `cloud.mongodb.com/v2/6463c2a911451523b69c6986#/clusters`. The page title is "Database Deployments" for "Cluster0". A red box highlights the "Browse Collections" button in the top navigation bar. The interface includes a sidebar with navigation options: "Database", "Services", and "Security". The main content area displays performance metrics and a table of deployment details.

Performance Metrics:

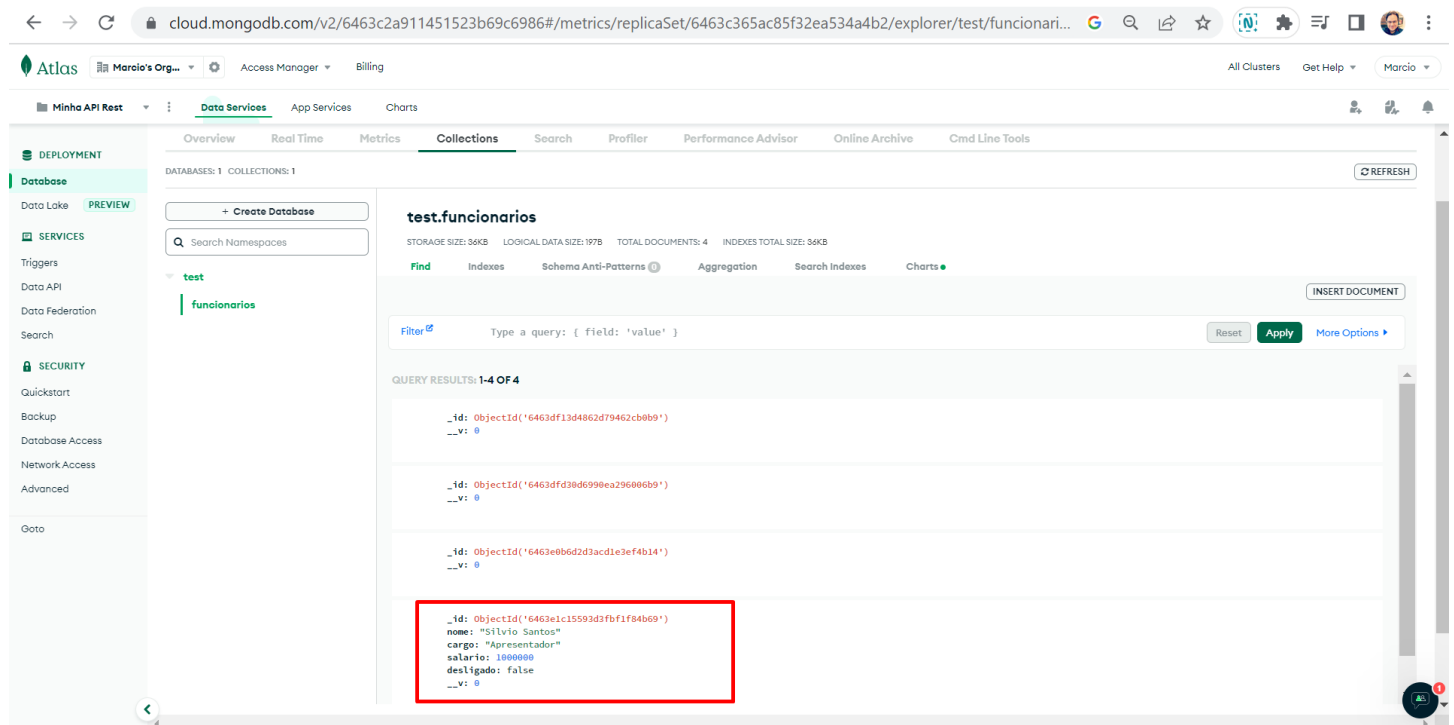
- Connections: 6.0
- In: 99.2 B/s
- Out: 215.0 B/s
- Data Size: 0.0 B

Deployment Details Table:

VERSION	REGION	CLUSTER TIER	TYPE	BACKUPS	LINKED APP SERVICES	ATLAS SQL	ATLAS SEARCH
6.0.6	AWS / N. Virginia (us-east-1)	M0 Sandbox (General)	Replica Set - 3 nodes	Inactive	None Linked	Connect	Create Index

System Status: All Good
©2023 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

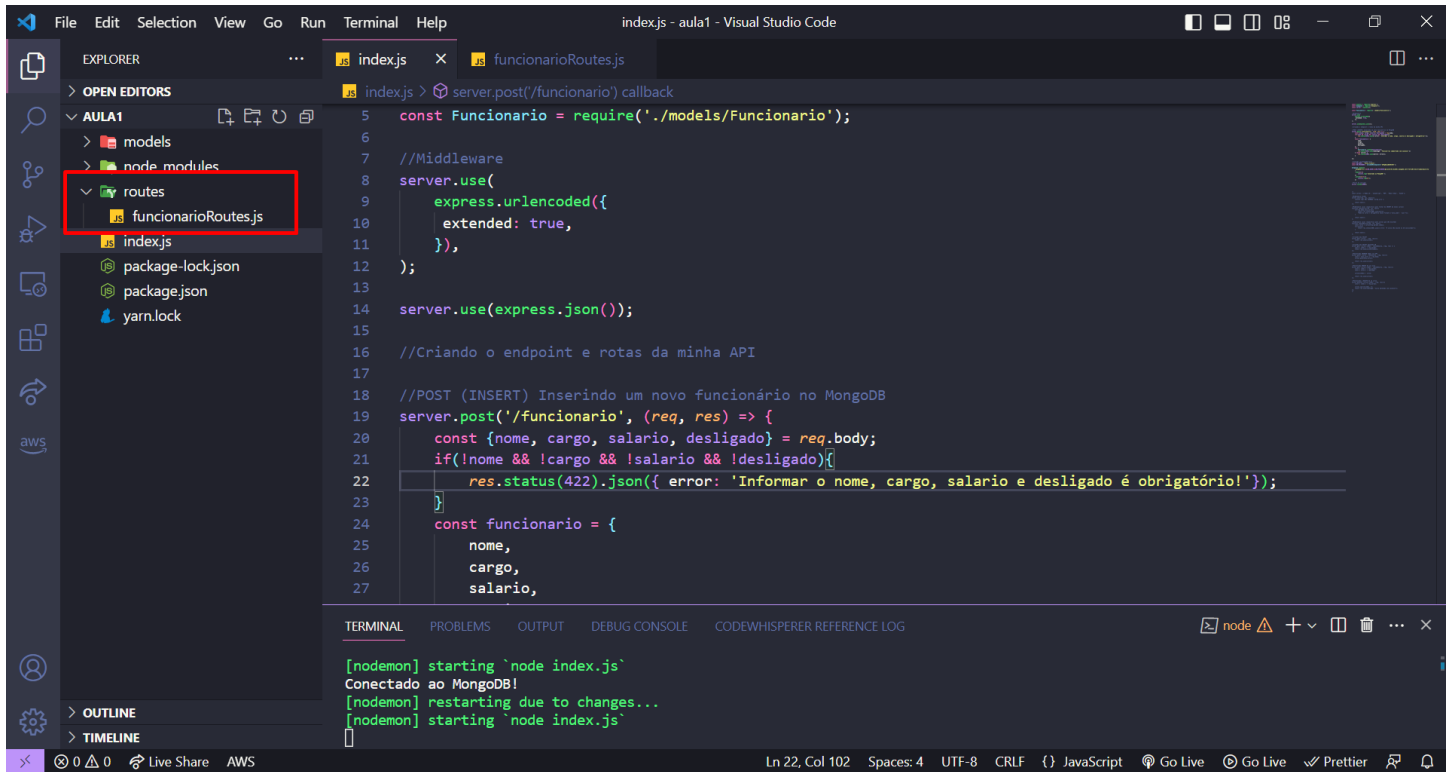
39 – Sim, o dado foi salvo no banco. Temos alguns registros vazios pois testamos antes de validar o body, podemos apagar os registros vazios.



The screenshot shows the MongoDB Atlas web interface. The left sidebar contains navigation options: DEPLOYMENT, Database (with a PREVIEW button), SERVICES, and SECURITY. The main content area is titled 'test.funcionarios' and shows a list of documents. The first three documents are empty, and the fourth document is highlighted in a red box. The document contains the following fields:

```
{
  "_id": ObjectId("6463e1c15593d3fbf1f84b69"),
  "nome": "Silvio Santos",
  "cargo": "Apresentador",
  "salario": 1000000,
  "desligado": false
}
```


40 – Para evitar de misturar conexão de banco com rotas, vamos agora organizar todas as rotas da API de funcionário separadamente do index.js. Crie uma pasta chamada funcionarioRoutes.js. No futuro nossa API pode ter rotas de cliente, por exemplo, podemos seguir essa mesma forma de organização de rotas.

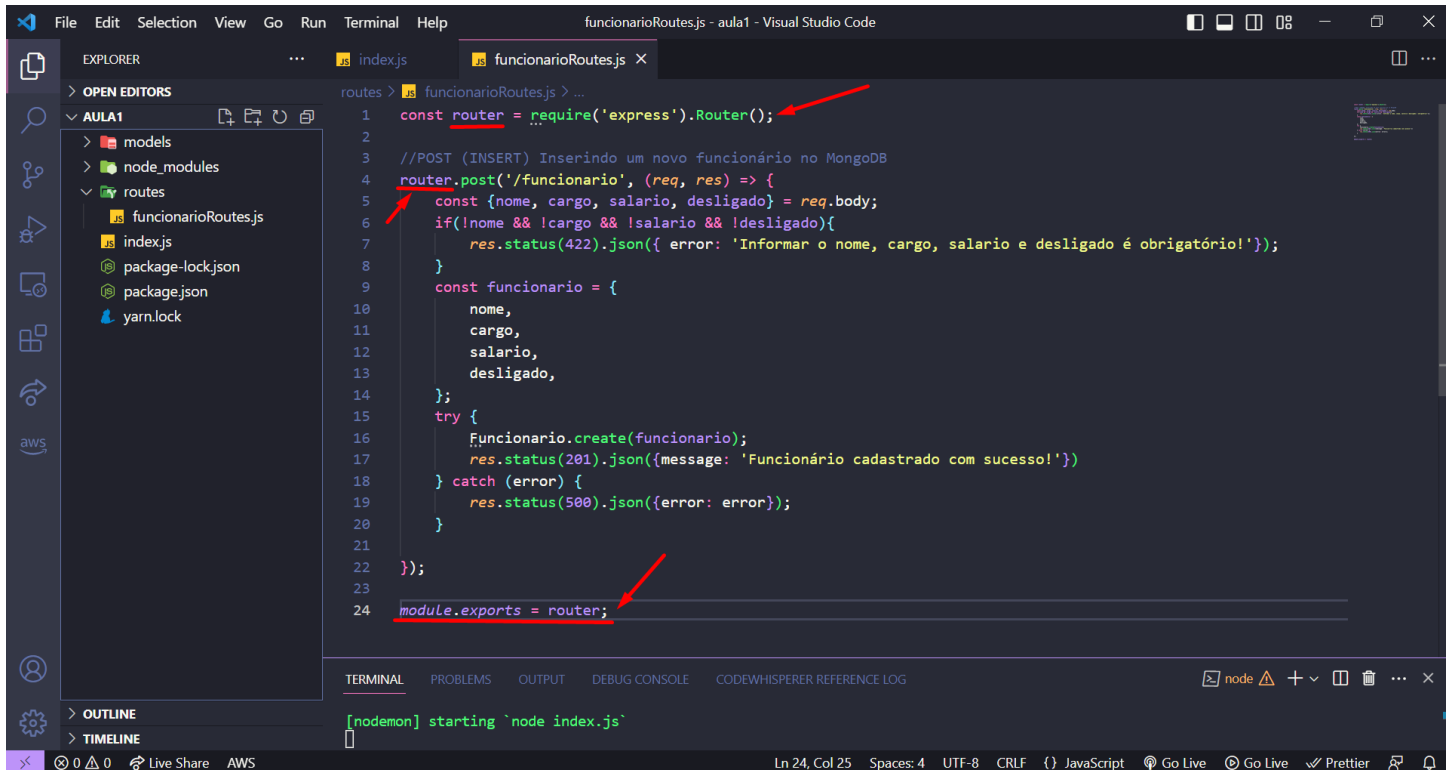


```
5 const Funcionario = require('./models/Funcionario');
6
7 //Middleware
8 server.use(
9   express.urlencoded({
10     extended: true,
11   }),
12 );
13
14 server.use(express.json());
15
16 //Criando o endpoint e rotas da minha API
17
18 //POST (INSERT) Inserindo um novo funcionário no MongoDB
19 server.post('/funcionario', (req, res) => {
20   const {nome, cargo, salario, desligado} = req.body;
21   if(!nome && !cargo && !salario && !desligado){
22     res.status(422).json({ error: 'Informar o nome, cargo, salario e desligado é obrigatório!'});
23   }
24   const funcionario = {
25     nome,
26     cargo,
27     salario,
```

Terminal output:

```
[nodemon] starting `node index.js`
Conectado ao MongoDB!
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
```

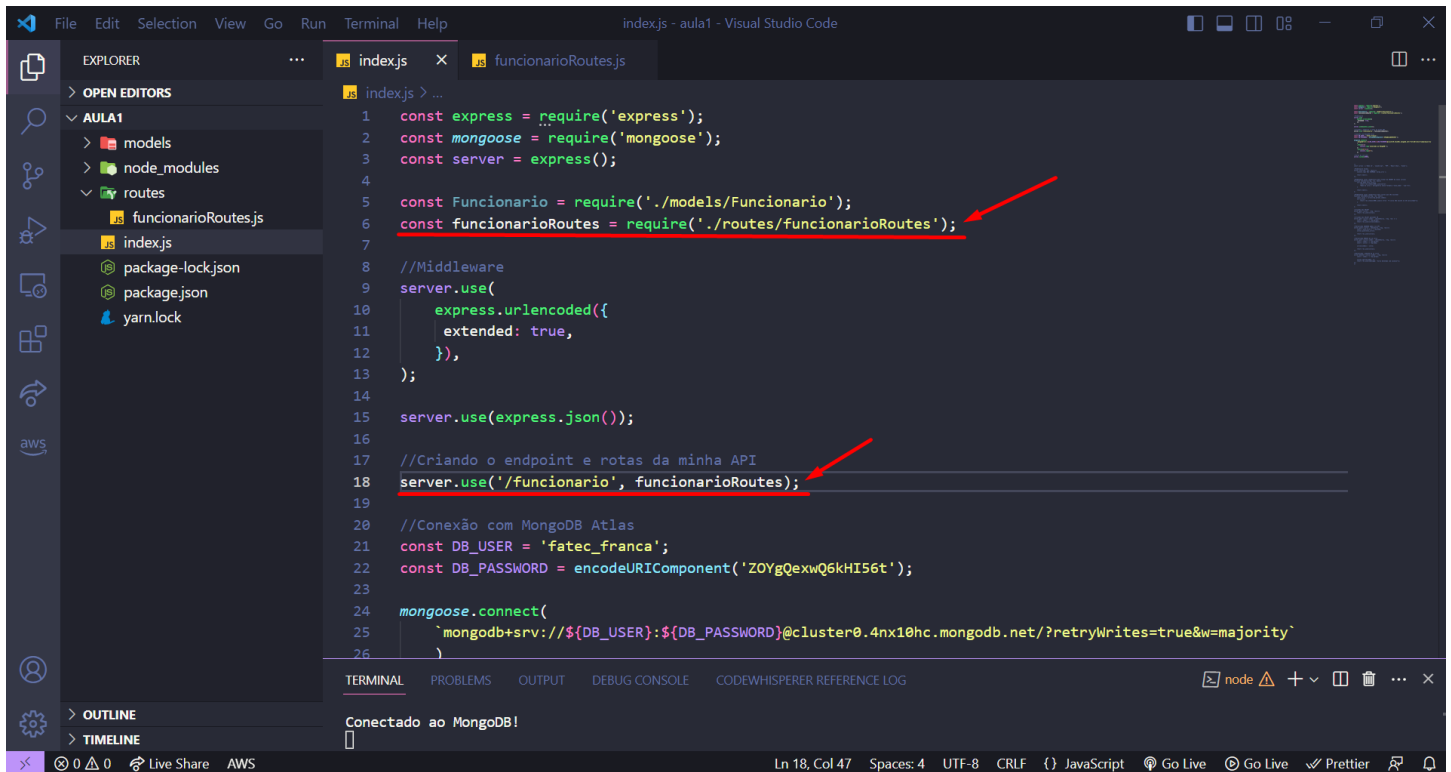
41 – Selecione todo o POST com a rota “/funcionario” recorte e cole no arquivo funcionarioRoutes.js. Agora criamos uma const chamada router que utiliza as funções de Router do express e onde usavamos server agora usamos router. Não se esqueça de exportar esse módulo.



```
1 const router = require('express').Router();
2
3 //POST (INSERT) Inserindo um novo funcionario no MongoDB
4 router.post('/funcionario', (req, res) => {
5   const {nome, cargo, salario, desligado} = req.body;
6   if(!nome && !cargo && !salario && !desligado){
7     res.status(422).json({ error: 'Informar o nome, cargo, salario e desligado é obrigatório!'});
8   }
9   const funcionario = {
10     nome,
11     cargo,
12     salario,
13     desligado,
14   };
15   try {
16     funcionario.create(funcionario);
17     res.status(201).json({message: 'Funcionário cadastrado com sucesso!'})
18   } catch (error) {
19     res.status(500).json({error: error});
20   }
21 }
22 });
23
24 module.exports = router;
```

Ln 24, Col 25 Spaces: 4 UTF-8 CRLF {} JavaScript Go Live Go Live Prettier

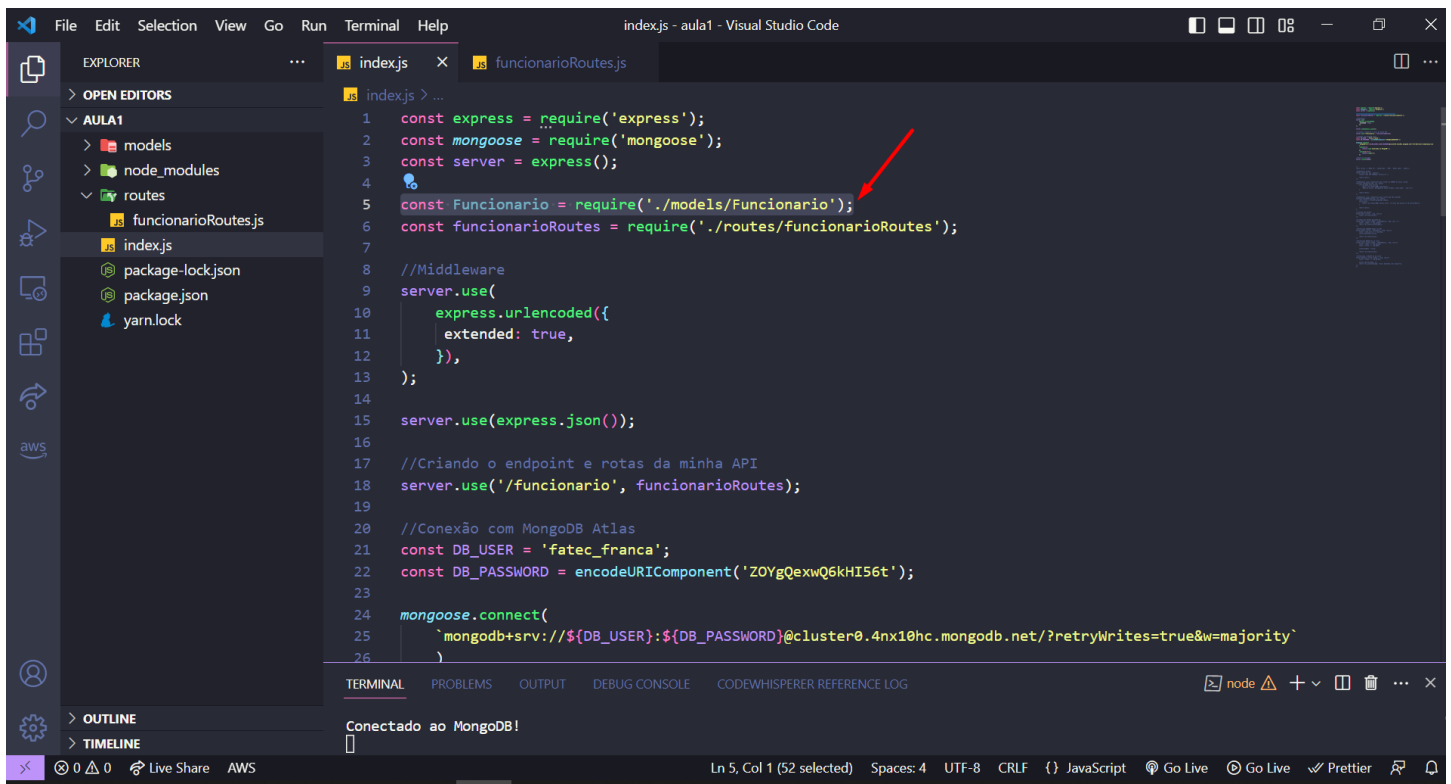
42 – Precisamos agora importar para nossa aplicação essa configuração nova de rotas. Importe o funcionarioRoutes passando o caminho do arquivo e depois na linha 18 estamos fazendo um redirecionamento de rotas, ou seja, quem chamar a API na rota “/funcionario” será automaticamente redirecionado para o arquivo funcionarioRoutes lidar com a requisição.



```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const server = express();
4
5 const Funcionario = require('./models/Funcionario');
6 const funcionarioRoutes = require('./routes/funcionarioRoutes');
7
8 //Middleware
9 server.use(
10   express.urlencoded({
11     extended: true,
12   }),
13 );
14
15 server.use(express.json());
16
17 //Criando o endpoint e rotas da minha API
18 server.use('/funcionario', funcionarioRoutes);
19
20 //Conexão com MongoDB Atlas
21 const DB_USER = 'fatec_franca';
22 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
23
24 mongoose.connect(
25   `mongodb+srv://${DB_USER}:${DB_PASSWORD}@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority`
26 )
```

Terminal output: Conectado ao MongoDB!

43 – Não precisamos mais do modelo Funcionario dentro do index.js. Recorte ele e cole no arquivo funcionarioRoutes.js.

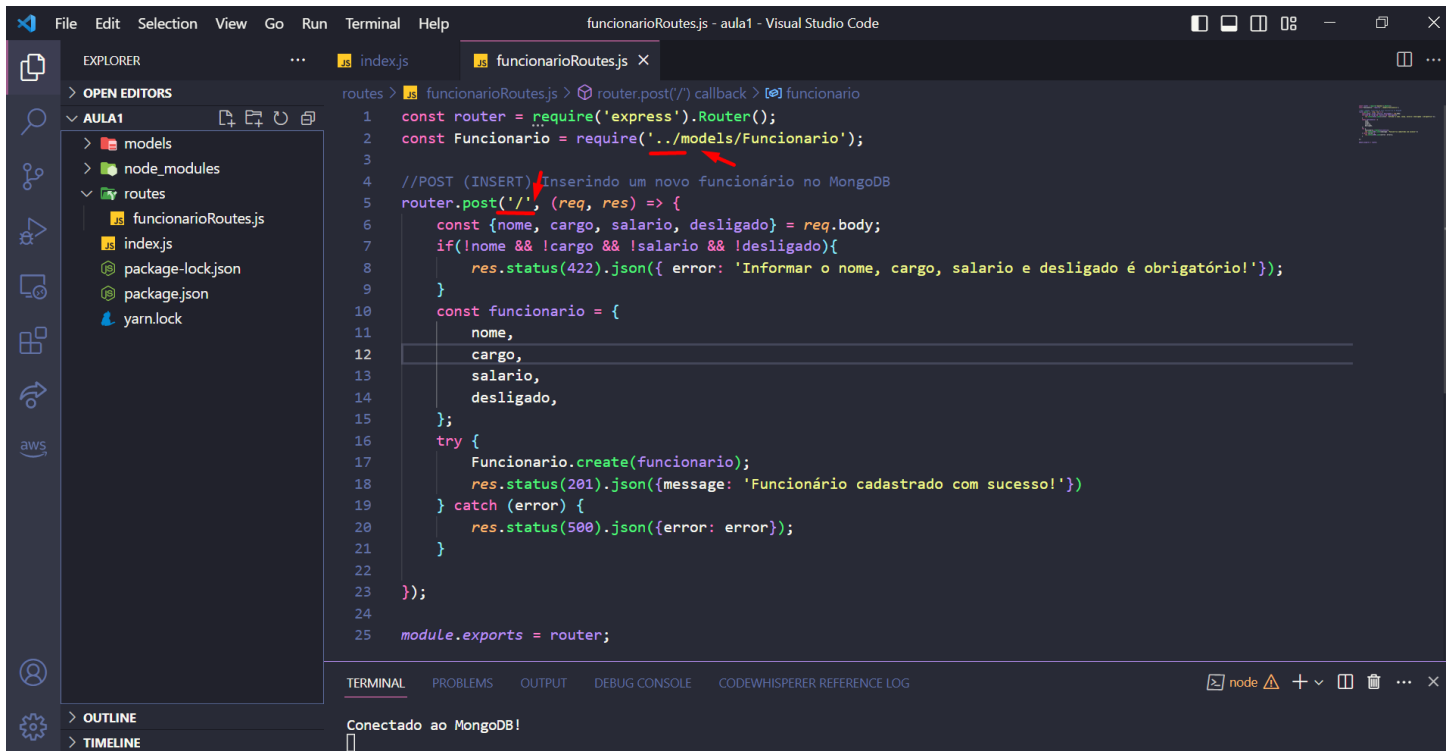


The screenshot shows the Visual Studio Code interface with the following details:

- EXPLORER:** The file explorer on the left shows a project structure with folders 'models' and 'routes', and files 'funcionarioRoutes.js', 'index.js', 'package-lock.json', 'package.json', and 'yarn.lock'.
- EDITOR:** The 'index.js' file is open, showing JavaScript code. A red arrow points to the line `const Funcionario = require('./models/Funcionario');` on line 5, which is being selected for a cut operation.
- Code in index.js:**

```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const server = express();
4
5 const Funcionario = require('./models/Funcionario');
6 const funcionarioRoutes = require('./routes/funcionarioRoutes');
7
8 //Middleware
9 server.use(
10   express.urlencoded({
11     extended: true,
12   }),
13 );
14
15 server.use(express.json());
16
17 //Criando o endpoint e rotas da minha API
18 server.use('/funcionario', funcionarioRoutes);
19
20 //Conexão com MongoDB Atlas
21 const DB_USER = 'fatec_franca';
22 const DB_PASSWORD = encodeURIComponent('ZOYgQexwQ6kHI56t');
23
24 mongoose.connect(
25   `mongodb+srv://${DB_USER}:${DB_PASSWORD}@cluster0.4nx10hc.mongodb.net/?retryWrites=true&w=majority`
26 )
```
- TERMINAL:** The terminal at the bottom shows the message 'Conectado ao MongoDB!'.
- Status Bar:** The bottom status bar indicates 'Ln 5, Col 1 (52 selected)', 'Spaces: 4', 'UTF-8', 'CRLF', and 'JavaScript'.

44 – Agora no arquivo `funcionarioRoutes.js` temos a importação do modelo `Funcionario`, não se esqueça de adicionar mais um ponto para que ele encontre o caminho correto. Também não precisamos mais dentro de `router.post` passar a rota `"/funcionario"` pois o nosso `index.js` já recebe essa rota e redireciona a requisição para esse arquivo.



The screenshot shows the Visual Studio Code editor with the file `funcionarioRoutes.js` open. The Explorer sidebar on the left shows the project structure with folders `models` and `node_modules`, and a `routes` folder containing `funcionarioRoutes.js`, `index.js`, `package-lock.json`, `package.json`, and `yarn.lock`. The main editor displays the following JavaScript code:

```
1 const router = require('express').Router();
2 const Funcionario = require('../models/Funcionario');
3
4 //POST (INSERT) Inserindo um novo funcionário no MongoDB
5 router.post('/', (req, res) => {
6   const {nome, cargo, salario, desligado} = req.body;
7   if(!nome && !cargo && !salario && !desligado){
8     res.status(422).json({ error: 'Informar o nome, cargo, salario e desligado é obrigatório!'});
9   }
10   const funcionario = {
11     nome,
12     cargo,
13     salario,
14     desligado,
15   };
16   try {
17     Funcionario.create(funcionario);
18     res.status(201).json({message: 'Funcionário cadastrado com sucesso!'})
19   } catch (error) {
20     res.status(500).json({error: error});
21   }
22 });
23
24 module.exports = router;
```

Red arrows in the original image point to the `require('../models/Funcionario')` line and the `'/'` path in the `router.post` call. The bottom status bar shows the terminal output: `Conectado ao MongoDB!`.

45 – Agora vamos testar se nossa nova estrutura de chamada API funciona bem, insira um novo funcionário via API.

The screenshot shows the Postman interface with a workspace named 'Fatec'. The left sidebar contains a 'Collections' panel with a collection named 'API NodeJS + MongoDB'. Inside this collection, there is a folder 'Testando API' and a test named 'Inserindo Funcionarios'. The main panel shows the details of this test, which is a POST request to 'localhost:3000/funcionario'. The request body is in JSON format and contains the following data:

```
1 {
2   "nome": "Hebe Camargo",
3   "cargo": "Apresentadora",
4   "salario": 50000,
5   "desligado": true
6 }
```

The response is also in JSON format and contains the following data:

```
1 {
2   "message": "Funcionário cadastrado com sucesso!"
3 }
```

A red arrow points to the response message. The status bar at the bottom indicates '201 Created', '190 ms', and '290 B'.

46 – Podemos ver que o dado é salvo com sucesso no MongoDB!

The screenshot displays the MongoDB Atlas web interface. The left sidebar contains navigation menus for Deployment, Services, and Security. The main panel is titled 'test.funcionarios' and shows a 'Find' tab with a query filter. The query results section displays two documents. The first document is for 'Silvio Santos' and the second, highlighted with a red box, is for 'Nélio Camargo'.

URL: cloud.mongodb.com/v2/6463c2a911451523b69c6986#/metrics/replicaSet/6463c365ac85f32ea534a4b2/explorer/test/funcionari...

Atlas | Marcio's Org... | Access Manager | Billing | All Clusters | Get Help | Marcio

Minha API Rest | Data Services | App Services | Charts

Overview | Real Time | Metrics | Collections | Search | Profiler | Performance Advisor | Online Archive | Cmd Line Tools

DATABASES: 1 COLLECTIONS: 1

+ Create Database

Search Namespaces

test

funcionarios

STORAGE SIZE: 36KB LOGICAL DATA SIZE: 197B TOTAL DOCUMENTS: 4 INDEXES TOTAL SIZE: 36KB

Find | Indexes | Schema Anti-Patterns | Aggregation | Search Indexes | Charts

INSERT DOCUMENT

Filter Type a query: { field: 'value' } [Reset] [Apply] [More Options]

QUERY RESULTS: 1-2 OF 2

```
{
  "_id": ObjectId("6463e1c15593d3fbf1f94b69"),
  "nome": "Silvio Santos",
  "cargo": "Apresentador",
  "salario": 1000000,
  "desligado": false,
  "__v": 0
}
```

```
{
  "_id": ObjectId("6463e7470e4324d3b19adf44"),
  "nome": "Nélio Camargo",
  "cargo": "Apresentadora",
  "salario": 50000,
  "desligado": true,
  "__v": 0
}
```

Exercício

01 – Utilizando os conhecimentos adquiridos sobre criação de API em back-end Nodejs, faça o restante do CRUD:

- Método GET para listar todos os funcionários cadastrados no MongoDB.
- Método UPDATE para editar os dados existentes no MongoDB.
- Método DELETE para deletar os dados existentes no MongoDB.
- A documentação do uso de sua API